

Academia Technical Foundation Course Workbook

© 2023 Snowflake Computing Inc. All Rights Reserved

23L12

Contents

1 Introduction to Snowflake and Snowflake Objects 4
1.1 Log In . . . 4
1.2 Explore the Home Screen . . . 5
1.3 Set Up Your User Profile . . . 6
1.4 Create Folders and Worksheets . . . 7
1.5 Explore Worksheets . . . 8
1.6 Create Objects . . . 10
1.7 Key Takeaways . . . 11

2 Visualizations in Snowsight 12
2.1 Set Up for the Lab . . . 12
2.2 Analyze the Results and Data . . . 14
2.3 Create a Dashboard . . . 18
2.4 Add Tiles to a Dashboard . . . 19
2.5 Share Your Dashboard . . . 22
2.6 Key Takeaways . . . 24

3 Caching and Query Performance 25
3.1 Accessing and Navigating the Query Profile . . . 25
3.2 Metadata Cache . . . 35
3.3 Data Cache . . . 36
3.4 Partition Pruning . . . 38
3.5 Determine if Spillage is Taking Place . . . 40
3.6 Review the EXPLAIN Plan . . . 42
3.7 Summary . . . 44
3.8 Key takeaways . . . 44

4 Loading Structured Data 46
4.1 Loading Data From an External Stage Into a Table Using COPY INTO . . . 46
4.2 Load the *region.tbl* File . . . 47
4.3 Loading a GZip Compressed File on an External Stage Into a Table . . . 48
4.4 Validating Data Prior to Load . . . 49
4.5 Error Handling . . . 50
4.6 Key Takeaways . . . 53

5 Data Transformation During Data Loading 55
5.1 Transforming Data During Load . . . 55
5.2 Key Takeaways . . . 58

6 Exploring Semi-Structured JSON Data 59
6.1 Query Simple JSON Data Using dot notation syntax . . . 59

© 2023 Snowflake Computing Inc. All Rights Reserved

6.2 Query Nested JSON Data with Dot Notation 62
6.3 Use `FLATTEN` to Query JSON Data 64
6.4 Try it out! 69
6.5 Key Takeaways 78
6.6 Solution 79

7 Introduction to Monitoring Usage and Billing 81
7.1 Monitor Usage and Billing with the `ACCOUNT_USAGE` schema 82
7.2 Billing Metrics 84
7.3 Key Takeaways 87

© 2023 Snowflake Computing Inc. All Rights Reserved

1 Introduction to Snowflake and Snowflake Objects

The purpose of this lab is to familiarize you with Snowflake's Snowsight user interface. Specifically, you will learn how to use Snowsight worksheets to create and use Snowflake objects that you will use to run queries and in your day-to-day work.

Learning Objectives:

- How to navigate Snowsight to find the tools you'll need
- How to create and manage folders and worksheets
- How to set the context
- How to create warehouses, databases, schemas, and tables
- How to run a simple query

Scenario:

You are working for Snowbear Air, which is an airline that flies to fun destinations all over the world. You've been asked to create a few Snowflake objects in a development environment to test out your SQL statements. You will need to:

- Create a database and a schema
- Create a virtual warehouse
- Create a table and populate it with data
- Run some test queries

HOW TO COMPLETE THIS LAB

For the first part of this lab, you will need to use the PDF workbook supplied with your course materials. Follow the instructions in the PDF to log in and explore the home page.

For the second part of the lab, you will load your user-specific .SQL and complete the remainder of the lab steps inside a worksheet. The .SQL files contain instructions for completing the lab steps, as well as the SQL you will need to run.



Before getting started, make sure you have downloaded the lab files for this course to a convenient location and unzipped the archive. When you do this, you will see a folder named **USERS**. This folder contains sub-folders with usernames - locate your username and open that folder. These are the SQL files you will use throughout this course.

Let's get started!

1.1 Log In

The instructions below explain how to log in to Snowsight. If your instructor gives you different instructions, follow those instead.

© 2023 Snowflake Computing Inc. All Rights Reserved

1.1.1 Access the URL provided to you for this course.

You will be taken to a login screen for the lab account.

1.1.2 Enter your username and password.

Obtain these credentials from your instructor.

1.1.3 Change your password when prompted.

This is not an error - the lab environments are set up so that all users are asked to change their password the first time they log in.

1.2 Explore the Home Screen

When you first log in, you will be placed into the home screen, which will look similar to the screen below:

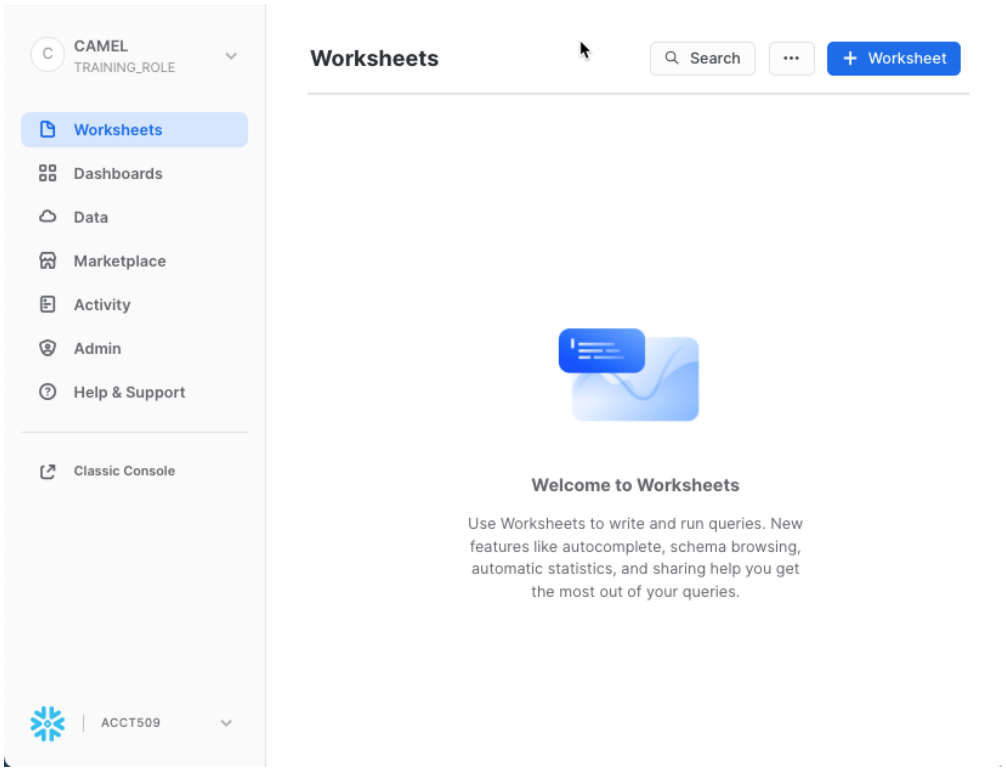


Figure 1: Home

1.2.1 Locate the user menu.

This is located in the upper left corner. Here you will find your username and the role that you currently have set. You will learn more about roles later on. You can always tell when you are in the Home Screen because the user

© 2023 Snowflake Computing Inc. All Rights Reserved

menu will be in the upper left.

1.2.2 Locate the left navigation bar.

This is located along the left side, beneath the user menu. You can use the navigation bar to move through various areas of the user interface.

1.2.3 Locate the account selector.

This is at the very bottom, on the left side. If you have accessed other Snowflake accounts, they will be listed here. You can click the selector to log in to a different account.

1.2.4 Locate the active panel.

This is the large area on the right. This will change based on which menu you have activated from the navigation bar.

1.3 Set Up Your User Profile

1.3.1 Access the profile.

Click the down arrow to the right side of the user menu, then select **Profile**.

1.3.2 Enter your name.

In the screen that appears, enter your name.



Changing your name in the **Profile** area does not change the username you enter to log into Snowsight. It simply changes your name in the display.

1.3.3 Optionally, enter your email address.

If you enter an email address, a verification link will be sent to the address you specify.

1.3.4 Set the **Default Experience** to Snowsight.

The default experience is probably already set to Snowsight, but change it if it is not. This determines whether you will be directed to the Classic UI or to Snowsight when you log in. In this course, we will be exclusively using Snowsight.

© 2023 Snowflake Computing Inc. All Rights Reserved



The Classic UI is going to be removed at some point in the future. Unless you need to access UI functionality that has not yet been ported to Snowsight, we recommend that you use Snowsight, so you are accustomed to it when the Classic UI is no longer available.

1.3.5 Save your profile changes.

Click **Save** when you are finished.

1.4 Create Folders and Worksheets

In this course, most of the work you do will be through worksheets. Worksheets can be created inside a folder for additional organization, but they do not NEED to be in folders. We will show you how to create folders if you want to use them.

1.4.1 Create a folder.

From the home screen, make sure you have **Worksheets** selected from the left navigation bar. Then click the ellipsis (three dots) that are just to the left of the blue **+ Worksheet** button (in the upper right corner of the screen).

From the menu that displays, select **New Folder**. Name your folder “Working with Snowsight” and then click **Create Folder**. You will now be positioned inside your empty folder.

1.4.2 Load your SQL file for this lab into a worksheet.

Click the down arrow that is just to the right of your folder name. Select **Create Worksheet from SQL File**. Then:

- Navigate to where you downloaded and unzipped the lab files.

- Open the **USERS** folder, and then open the sub-folder with your username.

- Open the file for this lab. A new worksheet will open with the SQL file you selected already loaded.

- Scroll to the top of the worksheet (when the file loads, your cursor will be positioned at the bottom of the worksheet, not the top).

- Follow the instructions using the PDF workbook for support!



In the SQL files, lab instructions are shown as comments. Do not skip these - they are instructions you need to follow. When the instructions indicate you need to run some SQL, we will provide the necessary code.

© 2023 Snowflake Computing Inc. All Rights Reserved

1.5 Explore Worksheets

Take a few minutes to familiarize yourself with the components of a worksheet. Each worksheet is a session connected to Snowflake. You can have many active worksheets (or sessions) at once. Each worksheet has a context, which defines the default components you are working on within this session.



> Your ***context*** is an important concept to understand. It defines the components that are being used by default in this session. It consists of a database, a schema, a role, and a virtual warehouse (you will learn more about all of these in the course of this class).
> - The role that is active will impact what objects you can see and what you can do with them.
> - The virtual warehouse defines what virtual warehouse you will use to run commands.
> - The database and schema define the default location you are acting in. For example, if you `SELECT * FROM <table>`, Snowflake will look for that table in the database and schema that is set in your context. If you want to select from a table that is outside your current context, you would need to provide the full path to that object, for example, `SELECT * FROM <database>.<schema>.<table>`.

1.5.1 Orient yourself to the worksheet components.

Spend a few minutes to orient yourself to the worksheet before beginning. Locate the following components:

- In the upper left corner, you will find a home icon. This will take you back to the home screen.

- To the right of the home icon is the name of your folder and your worksheet. Note that if you created the worksheet by loading a SQL file, the worksheet's name will match the name of the file you loaded. If you just created an empty worksheet, the name will be the current date and time.

- Directly below that is a filter button - just ignore this for now.

- Along the left side is the object browser. There are two tabs - one labeled **Worksheets** and the other labeled **Databases**. This identifies whether a list of your worksheets is displayed or if a list of database objects is displayed. You can switch between them to change what is displayed in the object browser.

- The large area on the right is the SQL editor pane. This is where you will type SQL commands. You should see this text loaded in the SQL editor panel.

- Just above the SQL pane, you will see a database name and a schema name. This sets the database and schema portion of your context.

- In the upper right corner, you will see a place where your role and virtual warehouse for this session are set. This sets the remainder of your context.

- To the right of that is a **Share** button - ignore that for now.

- On the far right is a circle with an arrow in it. This is the Run button.

© 2023 Snowflake Computing Inc. All Rights Reserved

1.5.2 Set your context.

Click on the arrow to the right of your **No Databsae Selected** in the SQL pane (part of your context). A list of current databases appears. Select **SNOWBEARAIR_DB** as your database and **MODELED** as your schema. Click outside the box to close it.

1.5.3 Create a virtual warehouse and set it in your context.

Before you can run any SQL commands, you need a virtual warehouse. Use the following SQL to create a virtual warehouse, and to set it in your context:

```
CREATE WAREHOUSE [login]_wh WITH
  WAREHOUSE_SIZE = XSmall
  INITIALLY_SUSPENDED = true;

USE WAREHOUSE [login]_wh;
```

If you look in the role/warehouse part of your context, you will now see **TRAINING_ROLE** as your role and [login]_WH as your virtual warehouse.

1.5.4 Show the tables that are in your current context.

```
SHOW TABLES;
```

Note that a `SHOW` command is a metadata operation - so you did not use your virtual warehouse. You can tell that the warehouse is suspended (not running) because the small dot to the left of the virtual warehouse name is gray.

1.5.5 Query some tables.

During this course, many of our exercises will use data from the **SNOWBEARAIR_DB** database. Take a look at some of that data:

```
SELECT *
FROM members
LIMIT 10;

SELECT TOP 5
    c.c_lastname,
    c.c_firstname,
    c.c_acctbal
FROM snowbearair_db.promo_catalog_sales.customer c
ORDER BY c.c_acctbal DESC;

SELECT
  c_firstname,
  c_lastname,
  o_orderkey,
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
  o_totalprice
FROM
  snowbearair_db.promo_catalog_sales.orders
JOIN
  snowbearair_db.promo_catalog_sales.customer
ON
  o_custkey = c_custkey
ORDER BY o_totalprice DESC
LIMIT 10;
```

The last query in that series shows how you can select from tables that are not in the database and schema that are currently set in your context.

Also, notice that these queries did require a virtual warehouse, so yours automatically started up - the dot next to the virtual warehouse name is now green. It will auto-suspend itself after some amount of idle time.

1.6 Create Objects

One of the things you may do during the course of your work is create objects, such as databases, schemas, and tables. Let's create a few objects that you will use during this course.

1.6.1 Create a database called `[login]_db` and set it in your context.

```
CREATE DATABASE [login]_db;

USE DATABASE [login]_db;
```

1.6.2 Create a schema called `my_schema` in the database you just created.

```
CREATE SCHEMA my_schema;

USE SCHEMA my_schema;
```

1.6.3 Set context defaults for this course.

By setting these defaults, you will ensure that these will be part of your context by default each time you open a worksheet in subsequent labs.

```
ALTER USER [login]

SET default_warehouse=[login]_wh
    default_namespace=[login]_db.public
    default_role=training_role;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

1.6.4 Create a temporary table.

```
CREATE TEMPORARY TABLE my_favorite_actors (name VARCHAR);
```

Now put a few rows in your table, substituting the placeholder names for the names of your actual favorite actors.

```
INSERT INTO my_favorite_actors
VALUES
  ('Heath Ledger'),
  ('Michelle Pfeiffer'),
  ('Meryl Street'),
  ('Anthony Hopkins'),
  ('Bruce Lee');

SELECT * FROM my_favorite_actors;
```

1.7 Key Takeaways

- You can create database objects both via the Snowsight UI and by executing SQL code in a worksheet. We did this exercise using SQL code.

- You can browse database objects and view their details by using the object browser in the worksheet.

- The context of a worksheet session consists of a role, schema, database, and virtual warehouse.

- The context of a worksheet can be set via the Snowsight UI or via SQL statements.

- You can create folders in which to save and organize worksheets.

© 2023 Snowflake Computing Inc. All Rights Reserved

2 Visualizations in Snowsight

The purpose of this lab is to show you how to use the visualization features and tools available in Snowsight. Specifically, you'll learn how to leverage our built-in contextual statistics columns in order to gain quick insights into data. Also, you'll learn how to create a dashboard from an existing query.

Learning Objectives:

- How to create a dashboard from a worksheet
- How to use ad-hoc filters to get insights into data
- How to add tiles to a dashboard
- How to share a dashboard

Scenario:

Snowbear Air is interested in seeing a year-by-year summary of gross sales. You've been asked to write a query with a graph and share it via a Snowflake dashboard. You've decided to use the PROMO_CATALOG_SALES schema to accomplish your task.

HOW TO COMPLETE THIS LAB

In the previous, lab you may have used the SQL code file for that lab to create a new worksheet and then just run the code provided within that worksheet. That approach will be modified a bit for this lab due to the nature of what you will be doing.



If you decide to use the SQL file to do this lab instead of the workbook PDF, note that there is a step in this lab that creates a dashboard from your worksheet. Once you do that step, you will lose access to the worksheet (and, thus, the instructions). Because of this, you will need to use the PDF workbook to complete this lab. The best way to complete this lab is to use the PDF workbook instructions for the entire lab from start to finish.

Let's get started!

2.1 Set Up for the Lab

2.1.1 Using skills you've already learned, create a new folder called "Visualizations".

2.1.2 Create a worksheet inside the Visualizations folder.

Click the down arrow just to the right of your folder name, and select **Create Worksheet from SQL File**. Navigate to the lab file and load it.

© 2023 Snowflake Computing Inc. All Rights Reserved

2.1.3 Rename the worksheet to “Dashboard Data.”

Click the down arrow just to the right of your worksheet’s default name (which is the name of the SQL file). Remove the default name and name your worksheet “Dashboard Data”.

2.1.4 Set the context by executing the statements below in your worksheet:

If you’re following along in your workbook PDF, you’ll notice that the file is filled with commented instructions. Just scroll down to the context commands shown below and execute them.

```
USE ROLE TRAINING_ROLE;
USE SCHEMA SNOWBEARAIR_DB.PROMO_CATALOG_SALES;
CREATE WAREHOUSE IF NOT EXISTS [login]_wh;
USE WAREHOUSE [login]_WH;
```

2.1.5 Run the query below:

```
SELECT
  YEAR(o.o_orderdate) AS year,
  SUM(l.l_extendedprice) AS sum_gross_revenue

FROM
        CUSTOMER C
        INNER JOIN ORDERS O ON C.C_CUSTKEY = O.O_CUSTKEY
        INNER JOIN LINEITEM L ON O.O_ORDERKEY = L.L_ORDERKEY

GROUP BY year
ORDER BY year;
```

You should see the results below:

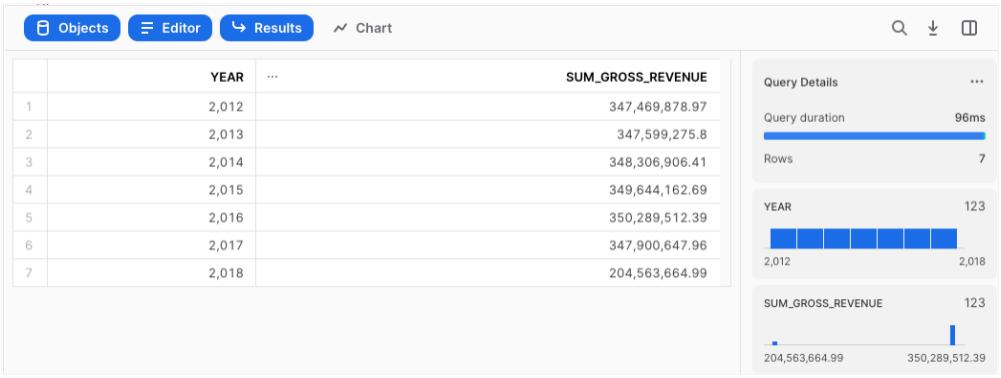


Figure 2: Query Results

2.1.6 Format the `YEAR` column.

Notice that both columns in the result have commas in them - but you don’t want commas in the `YEAR` column. Hover your mouse over the column name, and click the ellipsis (three dots) that appear on the left side of the

© 2023 Snowflake Computing Inc. All Rights Reserved

column header. A formatting panel will appear. Click the picture of the comma to remove commas from the values in the column, as shown below:

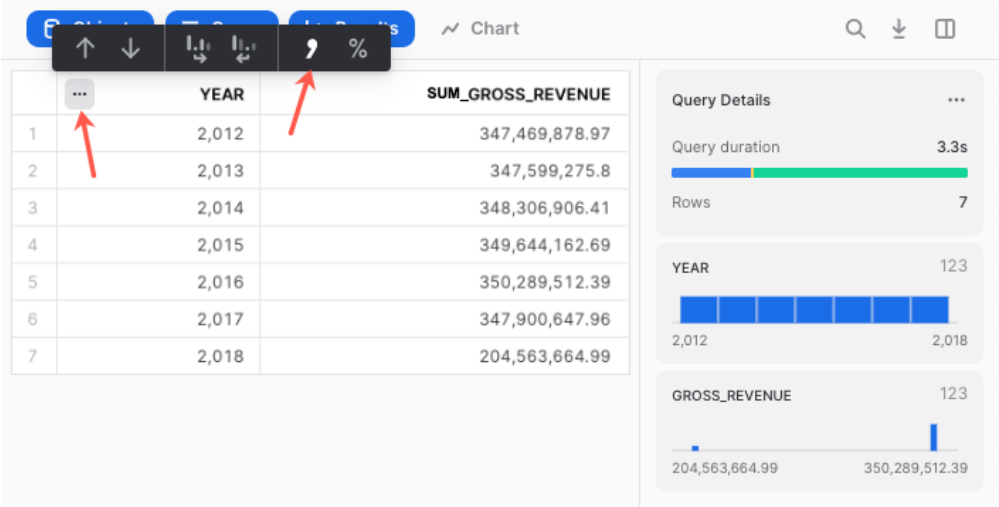


Figure 3: Removing comma from the YEAR column

You can click it again to put the commas back. You can also play with the other options to see what they do. When you are done, you should have no commas or decimal values in the YEAR column.

2.2 Analyze the Results and Data

2.2.1 Review the Query Details pane.

To the right of your query result, you will see a Query Details pane. Click on that pane (shown below) to see more information, such as the role and virtual warehouse used by the query.

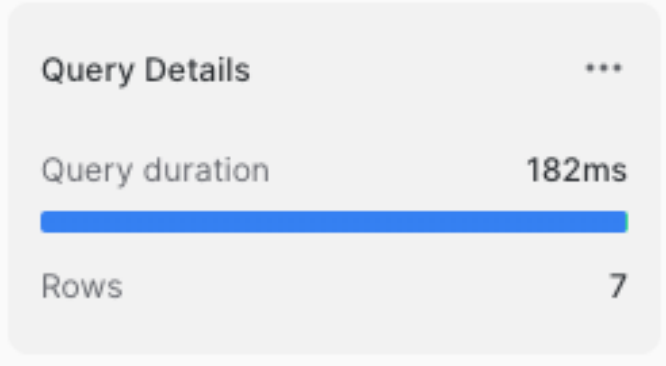


Figure 4: Query Details

© 2023 Snowflake Computing Inc. All Rights Reserved

2.2.2 View contextual statistics.

Scroll down. Below the Query Details pane, you will see contextual statistics for the two columns in your table: one pane labeled `YEAR` and the other labeled `SUM_GROSS_REVENUE`. The contextual statistics, one for each column returned by the query, can be used interactively as filters on the query result.

2.2.3 Activate the YEAR filter.

Click on the YEAR filter in the contextual statistics pane. This will also highlight the YEAR column in the query results, as shown below:

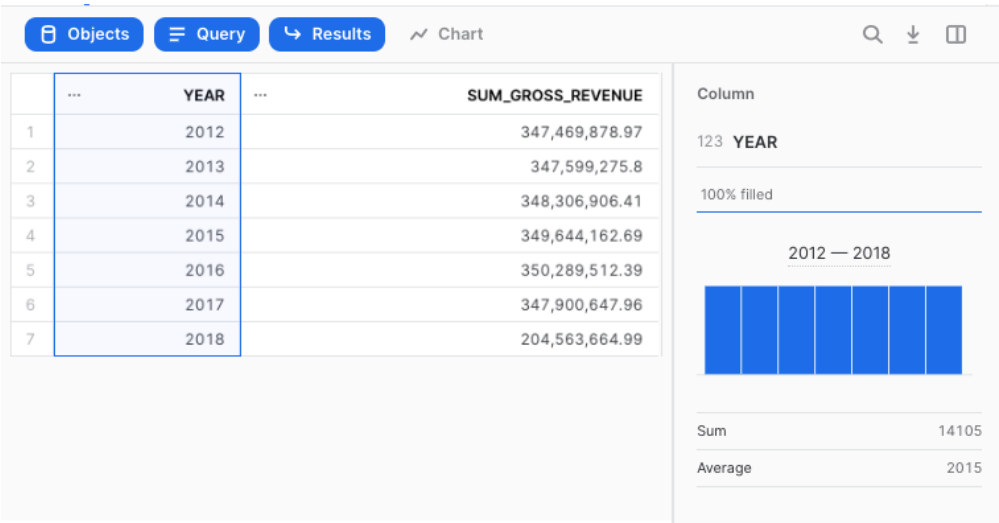


Figure 5: YEAR filter

2.2.4 Click on the leftmost column in the graph.

Now the results should be filtered for 2012 only:

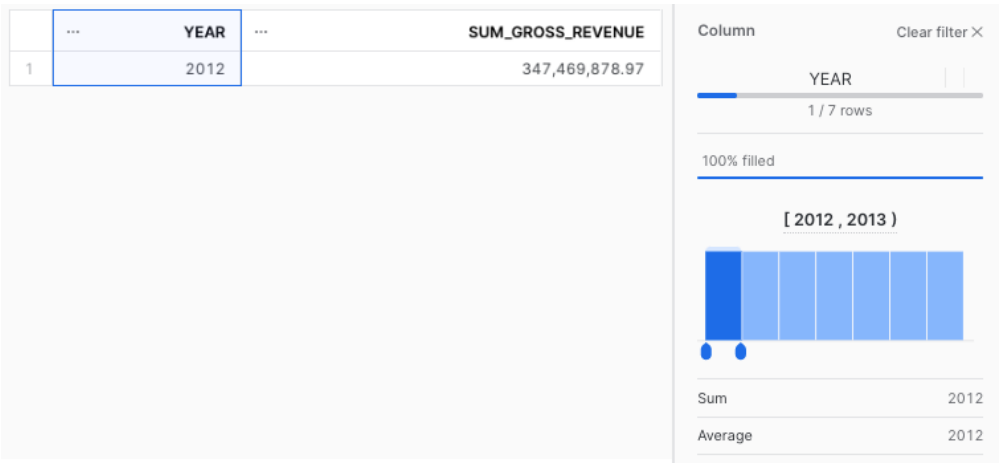


Figure 6: YEAR filter, 2012

© 2023 Snowflake Computing Inc. All Rights Reserved

2.2.5 Select 2012 and 2013.

Note the two oval selectors beneath the chosen column in the filter's graph. Click, hold, and drag the right-most selector to include both 2012 and 2013. Your filter should now appear as shown below:

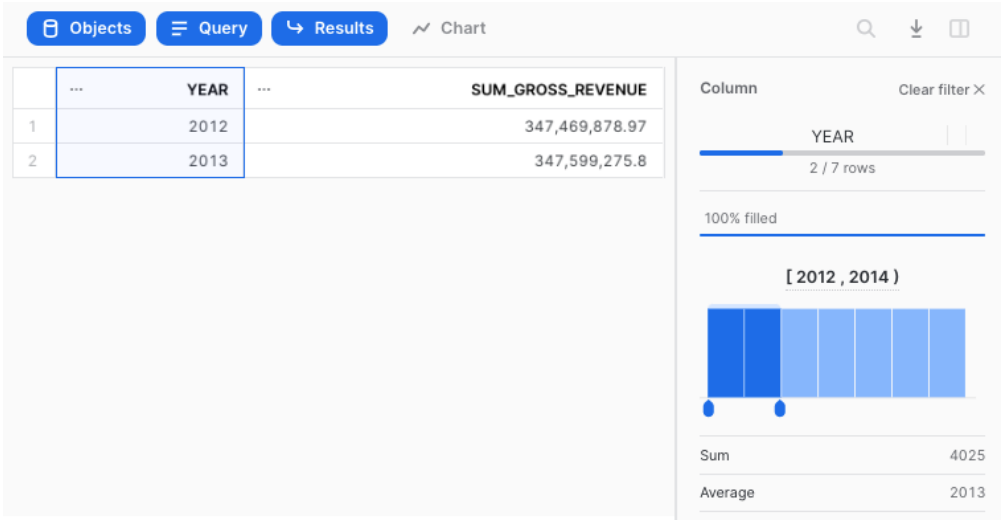


Figure 7: YEAR filter, 2012-2013

Now click different bars, or select any combination of multiple bars to see how the filter changes the data shown.

2.2.6 Clear the filter.

Click **Clear filter** at the top of the statistics pane.

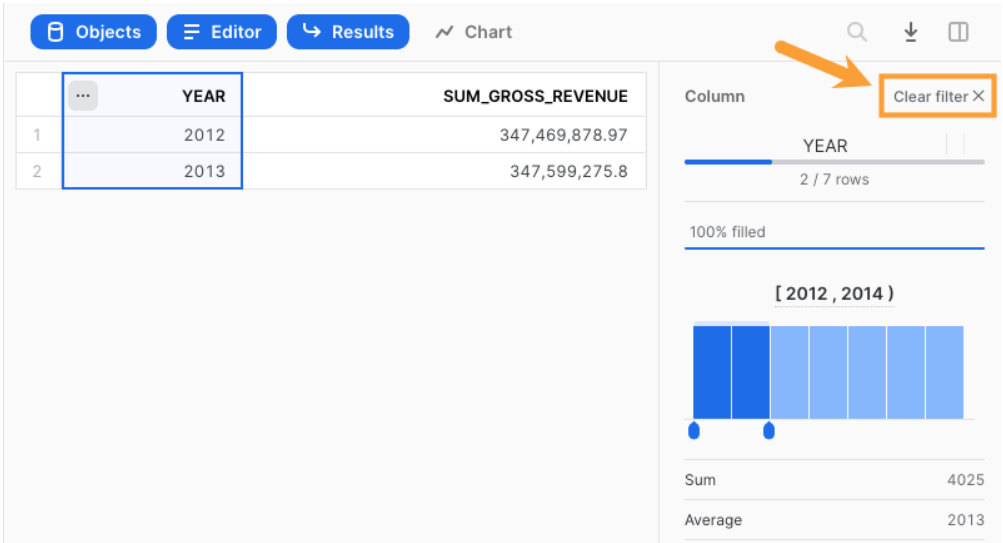


Figure 8: Clear filter button

© 2023 Snowflake Computing Inc. All Rights Reserved

This will return it to its unfiltered state.

2.2.7 Close the filter.

Click the X in the upper right corner of the panel. This will clear the column selected, and you should see the Query Details pane and the `YEAR` and `SUM_GROSS_REVENUE` filters.

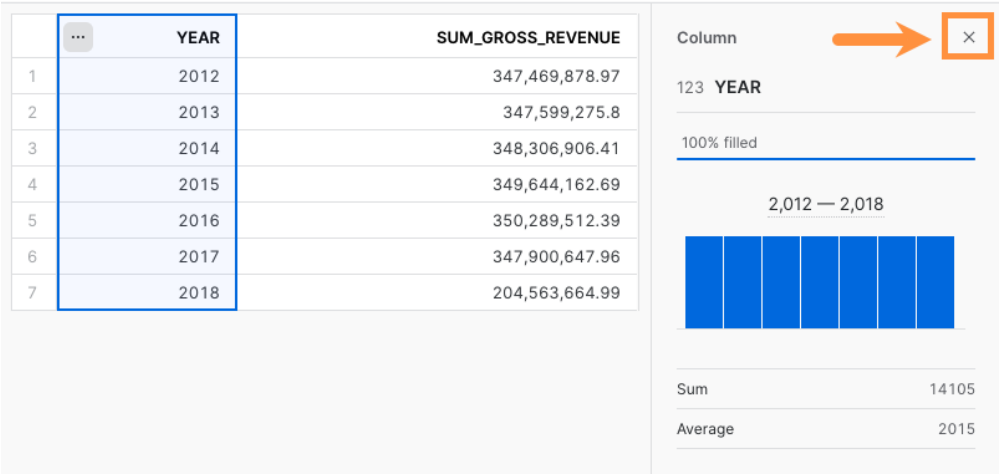


Figure 9: Clear Selection Button

2.2.8 Click the `SUM_GROSS_REVENUE` filter.

The filter should appear as below. Click each bar in the graph and observe how the data is filtered. Clicking between the bars will display the following message: “Query produced no results”. That’s because there is a gap between the value in the left-most bar and the value in the right-most bar.

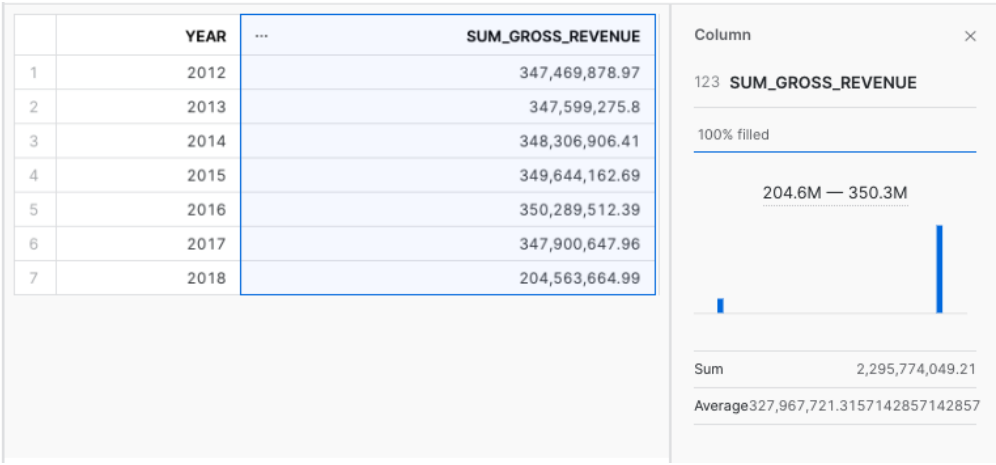


Figure 10: SUM_GROSS_REVENUE filter

© 2023 Snowflake Computing Inc. All Rights Reserved

2.2.9 Clear and close the filter.

Click **Clear filter** and then the **X** to close the filter.

2.3 Create a Dashboard

You can create a dashboard either from the dashboard area or by moving an existing worksheet to the dashboard area. The next step will move your worksheet out of the worksheet area into the dashboard area, so you will need access to the PDF for the instructions that follow.



Creating a dashboard from an existing worksheet **moves** the worksheet to the dashboard area (so it will no longer appear in your worksheets area). If you've been relying on the instructions in the SQL file, you will lose access to those instructions once you move the existing worksheet to a new dashboard. If you have not been using your PDF workbook thus far, open it now for the rest of the instructions.

2.3.1 Move your worksheet to the dashboard area.

Click the arrow to the right of the worksheet name, and navigate to **Move to** and then **+ New Dashboard**.

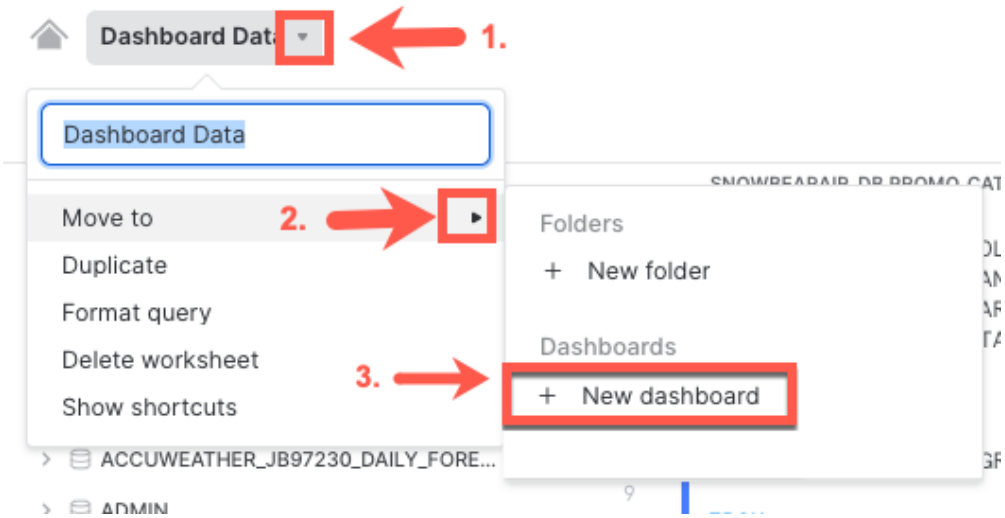


Figure 11: Move to Dashboards

2.3.2 Name the dashboard "[login] Gross Sales".

In the pop-up box that appears, enter your user name followed by "Gross Sales" as the dashboard name, and click **Create Dashboard**.

You will now see the worksheet that has been moved to the Dashboard area.

© 2023 Snowflake Computing Inc. All Rights Reserved

The worksheet will contain the instructions for this lab, but don't worry. Because they are commented out, they won't affect the result of your query, nor will they cause an error.

2.3.3 Rename the worksheet.

The name of the worksheet (shown at the top) will become the name of the tile in your dashboard. Change the name to "Gross Sales Data."

2.3.4 Click **- Return to [login] Gross Sales** in the upper left corner.

You should now see a dashboard tile that contains the results of the query.



Gross Sales Data — 7 rows

YEAR	SUM_GROSS_REVENUE
2012	347,469,878.97
2013	347,599,275.8
2014	348,306,906.41
2015	349,644,162.69
2016	350,289,512.39
2017	347,900,647.96
2018	204,563,664.99

Figure 12: Presentation mode

> Tiles are used to present data or graphs in a dashboard.

2.4 Add Tiles to a Dashboard

There are several ways to add a tile to an existing dashboard. You can create a new tile from scratch, or you can duplicate an existing tile to modify its query. Or, if you have a tile that contains numeric data and you want to create a graph of that data, you can simply edit the query for that tile and press the Chart button. We are going to use the third method since our new tile will just be a different representation of the same data.

2.4.1 Create a new tile.

In your dashboard, click the ellipsis (three dots) that appears in the upper far right corner of your existing **tile** (not dashboard) and select **Edit Query**. You will be taken back to the original query with the query result.

2.4.2 Press the Chart button just below the SQL and just above the query results.

A line graph will be displayed by default:

© 2023 Snowflake Computing Inc. All Rights Reserved

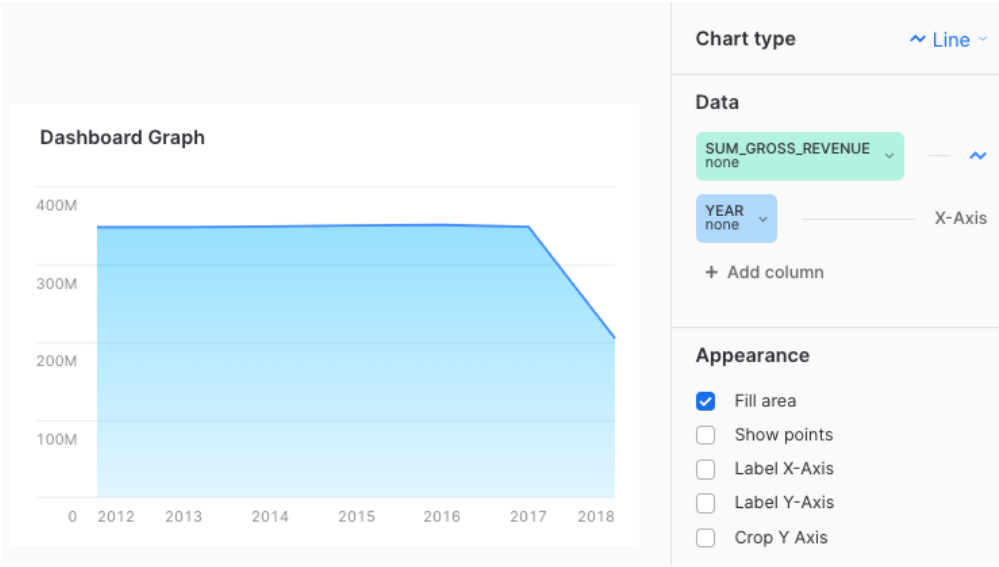


Figure 13: Presentation mode

2.4.3 Click **- Return to [login] Gross Sales** in the upper left.

You should now see a completed dashboard like the one shown below:

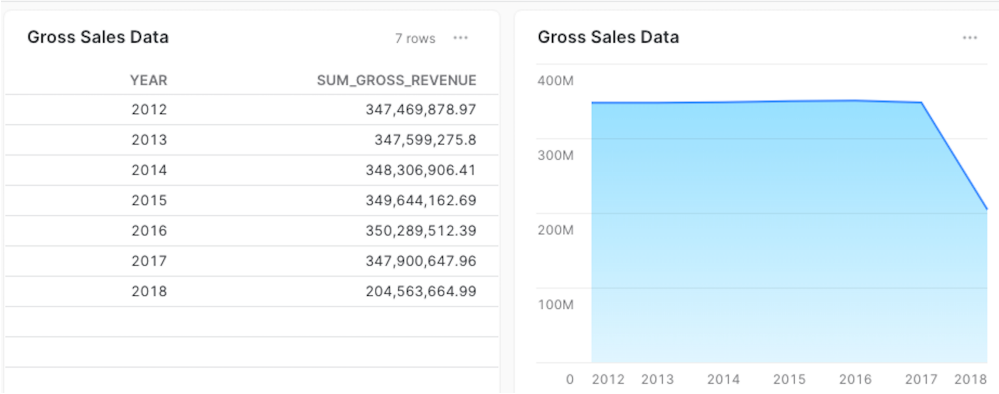


Figure 14: Completed Dashboard

2.4.4 Reposition the graph tile above the data tile.

Click, hold and drag the graph tile. When you see a blue cursor appear just above the tile with the gross sales table, release the mouse key.

© 2023 Snowflake Computing Inc. All Rights Reserved

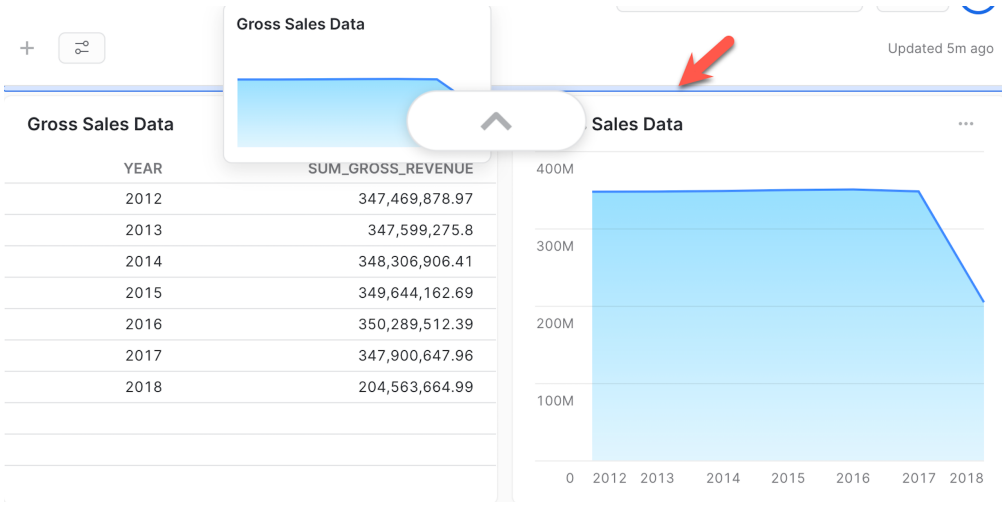


Figure 15: Repositioning a tile

Your dashboard should now appear as it does below:

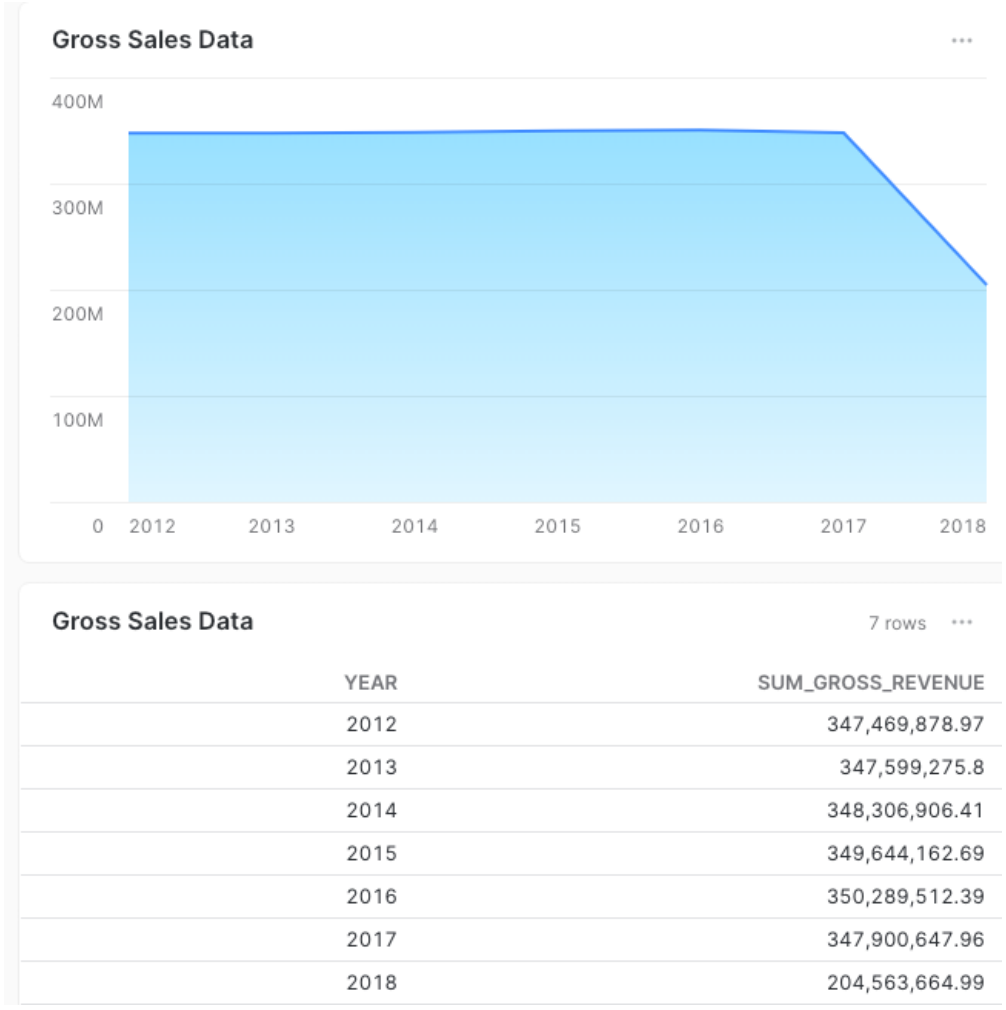


Figure 16: Final Dashboard

© 2023 Snowflake Computing Inc. All Rights Reserved

2.5 Share Your Dashboard

2.5.1 Click the **Share** button in the upper-right corner.

In this dialog box, you can search for someone and invite them to view and use this dashboard.

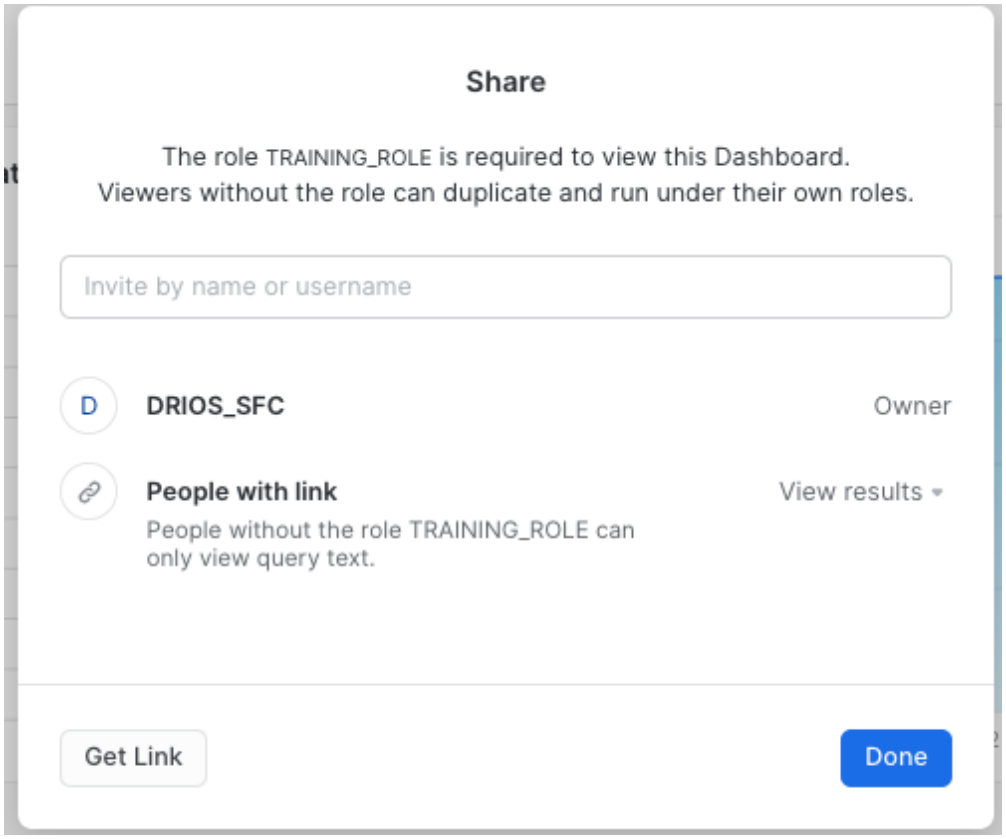


Figure 17: Share dialog box

© 2023 Snowflake Computing Inc. All Rights Reserved

2.5.2 Enter “Instructor1” as the user to share with.

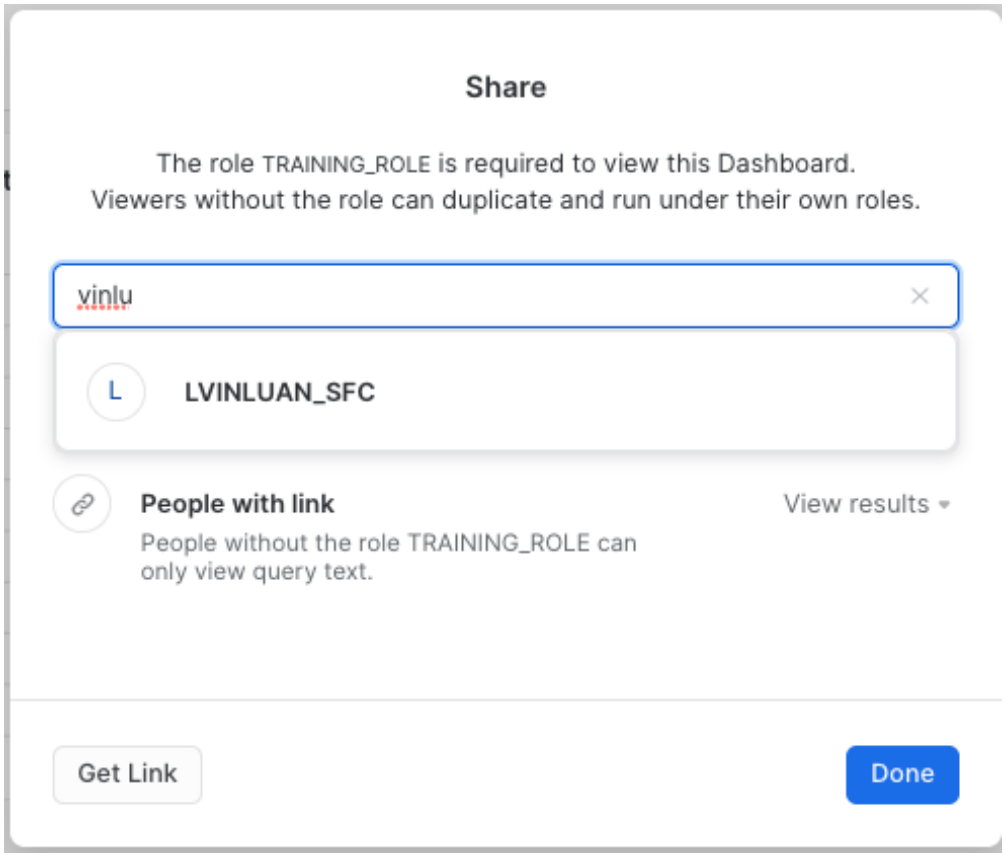


Figure 18: Sharing with a user

© 2023 Snowflake Computing Inc. All Rights Reserved

2.5.3 Set permissions to “View + run”.

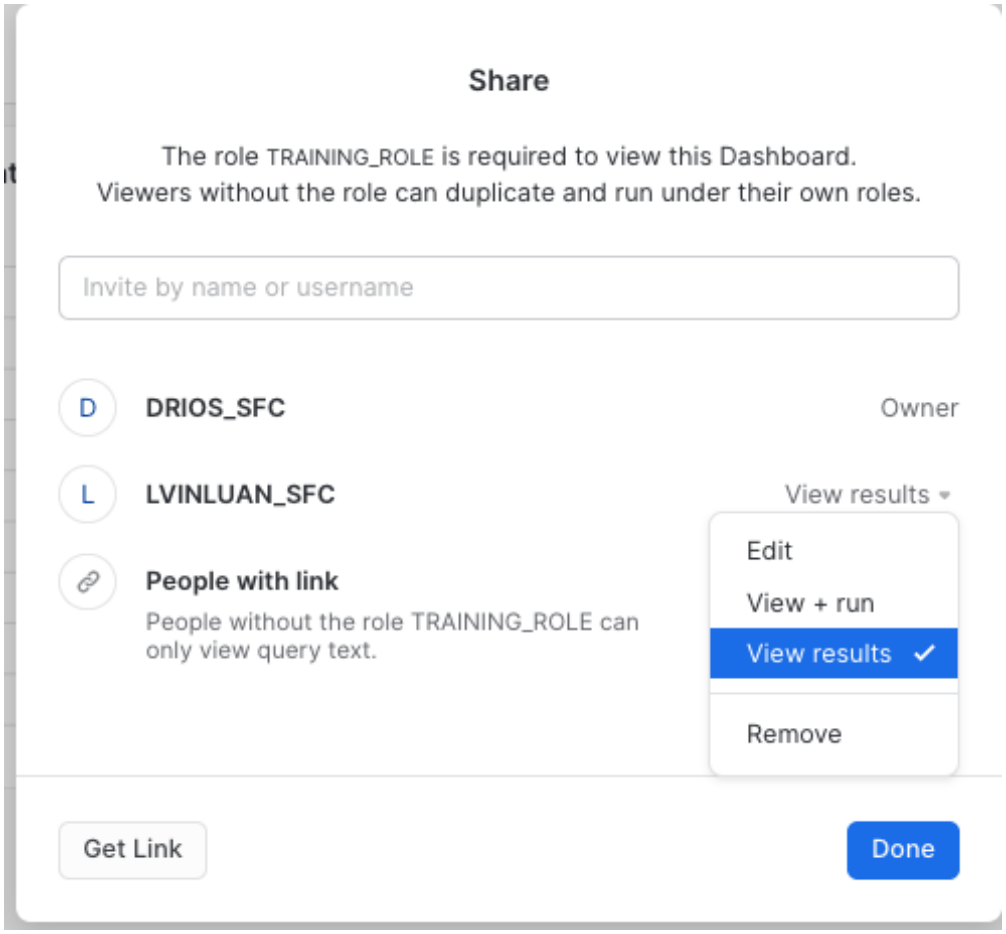


Figure 19: Granting permissions to user

Click the down arrow to the right of the user name, and select **View + run**. Then click **Done**. You have now shared your dashboard.

2.6 Key Takeaways

- While conducting ad-hoc analyses, you can use filters to gain insights into your data.

- You can create dashboards out of existing worksheets.

- Snowflake makes it super-easy to share worksheets with colleagues.

© 2023 Snowflake Computing Inc. All Rights Reserved

3 Caching and Query Performance

The purpose of this lab is to introduce you to the three types of caching Snowflake employs and how you can use the Query Profile to determine if your query is making the best use of caching.

Learning Objectives:

- How to access and navigate the Query Profile.
- Understand the differences between metadata cache, query result cache, and data cache.
- How to determine when and why the query result cache is being used or not.
- How to determine if partition pruning is efficient, and if not, how to improve it.
- How to determine if spillage is taking place.
- How to use EXPLAIN to determine how to improve your queries.

Scenario:

Like traditional relational database management systems, Snowflake employs caching to help you get the query results you want as fast as possible. Snowflake caching is turned on by default, and it works for you in the background without you having to do anything. However, if you aren't sure if you're writing queries that leverage caching in the most efficient way possible, you can use the Query Profile to determine how caching is impacting your queries.

Imagine that you're a data analyst at Snowbear Air and that you've just learned that Snowflake has different types of caching. You have been working on a few queries that you think could be running faster, but you're not sure. You've decided to become familiar with the Query Profile and plan to use it to see how caching is impacting your queries.

HOW TO COMPLETE THIS LAB

In order to complete this lab, you can type the SQL commands below directly into a worksheet. It is not recommended that you cut and paste from the workbook pdf as that sometimes results in errors.

You can also use the SQL code file for this lab that was provided at the start of the class. To open an .SQL file in Snowsight, make sure the Worksheet section is selected on the left navigation bar. Click on the ellipsis between the Search and +Worksheet buttons. In the dropdown menu, select "Create Worksheet from SQL File".

Let's get started!

3.1 Accessing and Navigating the Query Profile

In this section, you'll learn how to access, navigate and use the Query Profile. This will prepare you for analyzing query performance and caching in this lab.

© 2023 Snowflake Computing Inc. All Rights Reserved

3.1.1 Create a new folder and call it Caching and Query Performance.

3.1.2 Create a new worksheet inside the folder and call it Working with Cache and the Query Profile.

3.1.3 Set the worksheet context as shown below:

We'll also alter our session to avoid using cached results from previous sessions. This will allow us to see what the cache really is doing going forward.

```
USE ROLE TRAINING_ROLE;
USE WAREHOUSE [login]_WH;
CREATE DATABASE IF NOT EXISTS [login]_DB;
USE SCHEMA [login]_DB.PUBLIC;

--This will clear the data from previous sessions that is cached in the virtual
   warehouse
ALTER WAREHOUSE [login]_WH SUSPEND;
ALTER WAREHOUSE [login]_WH RESUME;
```

3.1.4 Create some data to query.

Here we'll create a table and populate it with data from the customer table. We'll also suspend the virtual warehouse to make sure our subsequent query doesn't just pull from cached data.

```
CREATE TABLE customer AS
SELECT
       c_custkey
     , c_firstname
     , c_lastname
  FROM
       SNOWBEARAIR_DB.PROMO_CATALOG_SALES.customer;

ALTER WAREHOUSE [login]_WH SUSPEND;
```

3.1.5 Execute the following simple query in your worksheet.

We need query results to view in the profile. Run the statement below.

```
SELECT DISTINCT
        *
FROM
       customer c;
```

3.1.6 Access the Query Profile.

Now let's view the profile. To the right of the query results, you will see a panel saying, 'Result limit exceeded….'. Click the 'X' in the upper right of this panel to show the usual Query Details panel. Click the ellipses shown in the screenshot. When the dialog box appears, click "View Query Profile". The Query Profile will open in a new tab.

© 2023 Snowflake Computing Inc. All Rights Reserved

Please note that what you may see might differ slightly from what is shown here. Since you are working in an environment that is not the same as the environment used to create the screenshots shown here, these differences are normal and to be expected.

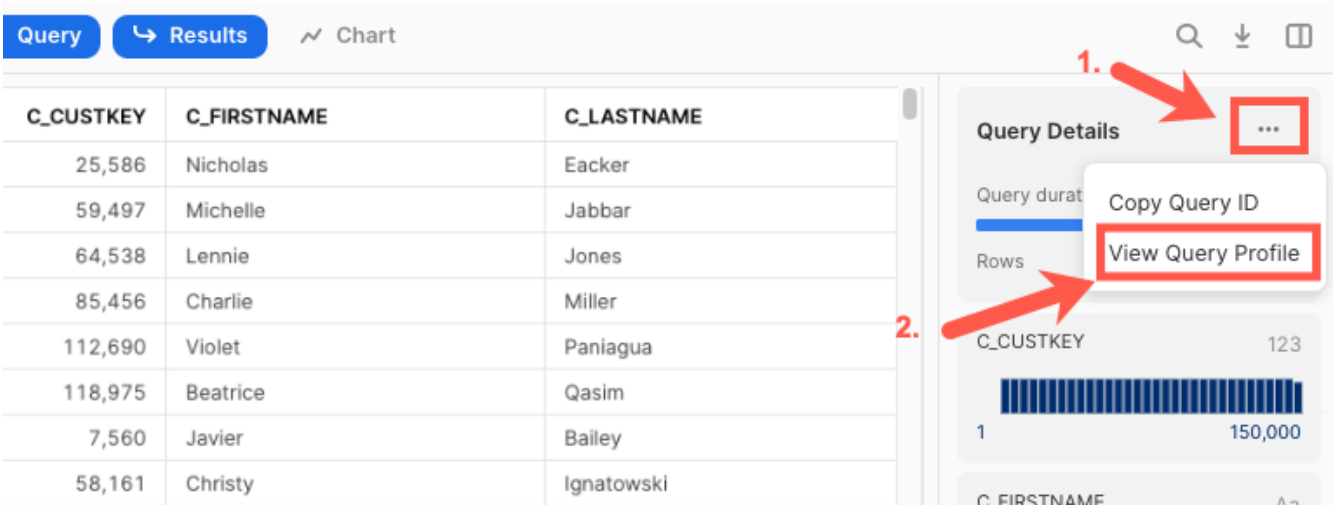


Figure 20: Accessing the Query Profile

The query profile will appear as shown below:

© 2023 Snowflake Computing Inc. All Rights Reserved

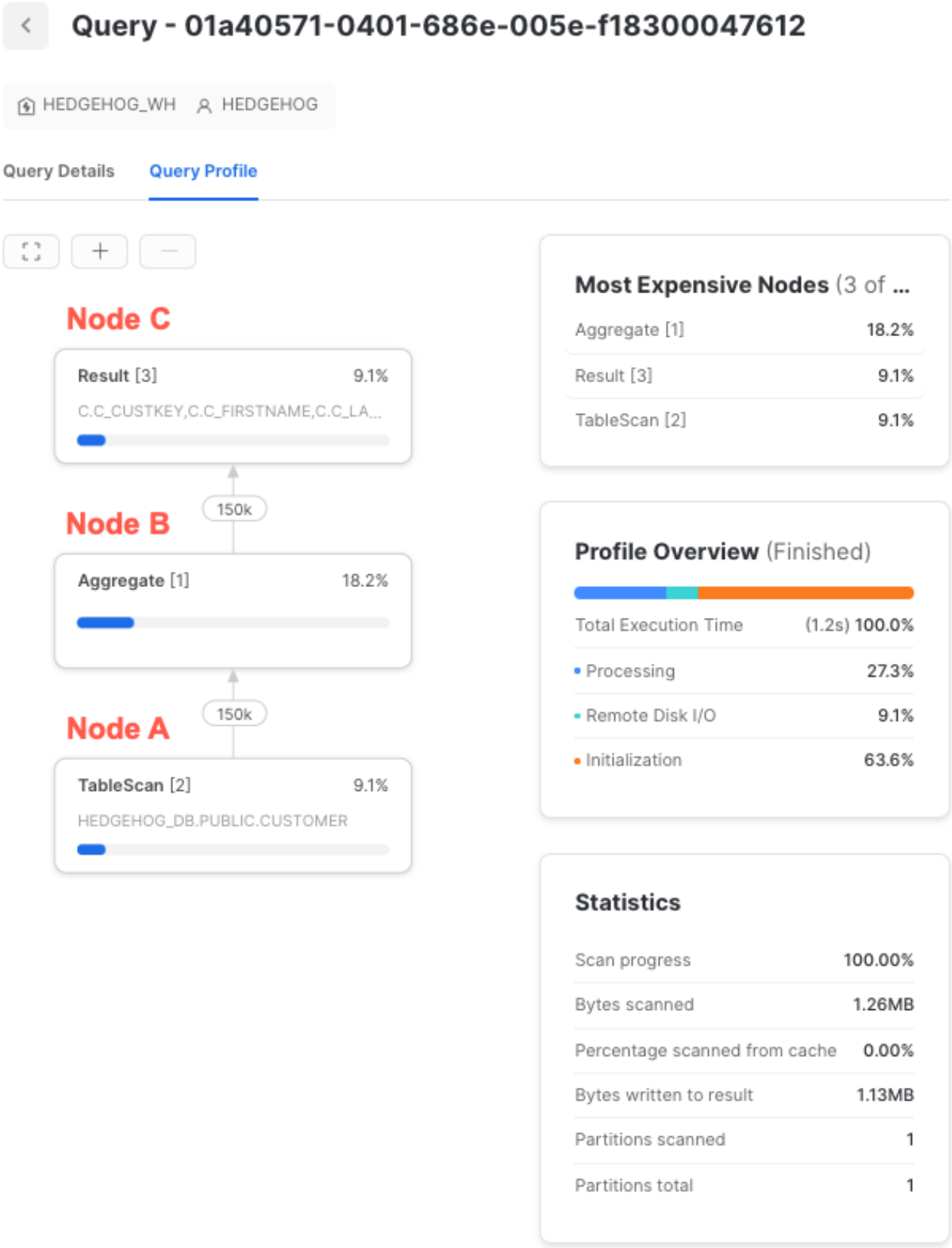


Figure 21: Query Profile

© 2023 Snowflake Computing Inc. All Rights Reserved

Note that there are two tabs in the header of the screen: Query Details and Query Profile.

3.1.7 Click the query details tab.

- The Query Details panel shows the status of execution, the overall execution duration, and other details. By looking at this, you can see if the query succeeded and if it ran within the time frame you were hoping for.

3.1.8 Click the query profile tab.

Note that there are two panels in the query profile that show specific aspects of execution.

NOTE Your results may differ from what is shown in the screenshots below.

- The Most Expensive Nodes panel shows the nodes that took the longest to execute. In this case, the three nodes all took the same amount of time. But if one node took a long time to execute, this panel lets you identify and analyze it with the purpose of making it run more efficiently.

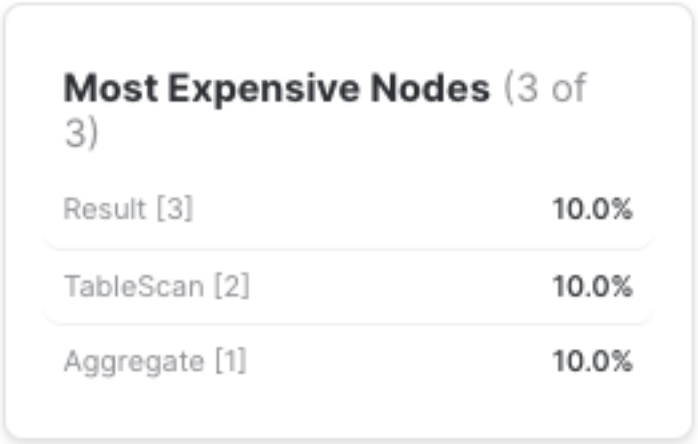


Figure 22: Most Expensive Nodes Panel

- The Profile Overview Panel displays the percentage of execution time that went towards the following:
  - Processing — The percentage of time spent on data processing by the CPU.
  - Remote Disk IO — The percentage of time when the processing was blocked by remote disk access.
  - Initialization — The percentage of time spent setting up the query processing.

Note that 70% of the execution time was spent setting up the query (Initialization), 10% was blocked by remote disk access (Remote Disk I/O), and 20% was spent on processing data for the query by the CPU (Processing). Most of the execution time (which was just slightly over 1 second) was spent in Initialization due to the fact we were running a very simple query against a small amount of data. If we had been running a more complex query against more data and the execution time was not satisfactory, finding a high disk I/O would tell us we need to do something to reduce that, such as filtering.

© 2023 Snowflake Computing Inc. All Rights Reserved

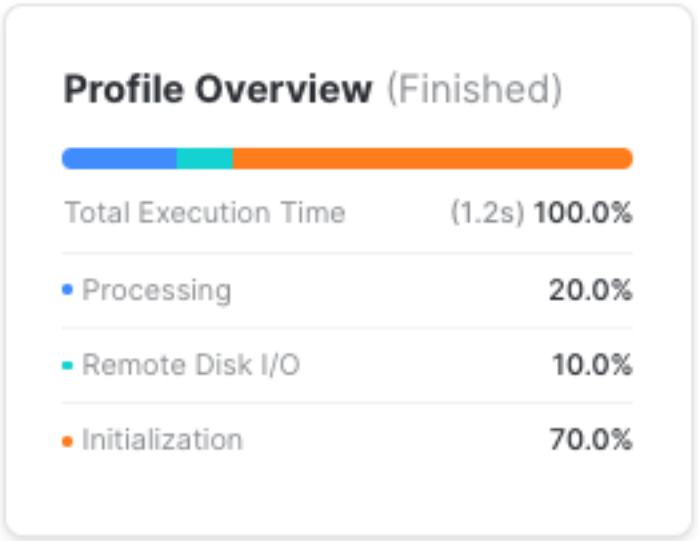


Figure 23: Profile Overview Panel

- The Statistics panel shows values related to the scanning of partitions. The critical fields are the number of partitions scanned and the number of total partitions. These figures tell you whether partition pruning is efficient or not. If the partitions scanned are a far smaller number than the total partitions, you're getting good pruning and should be getting a satisfactory execution time. If the numbers are similar or identical in cases where you expect pruning to take place, you may need to revisit and revise your query.

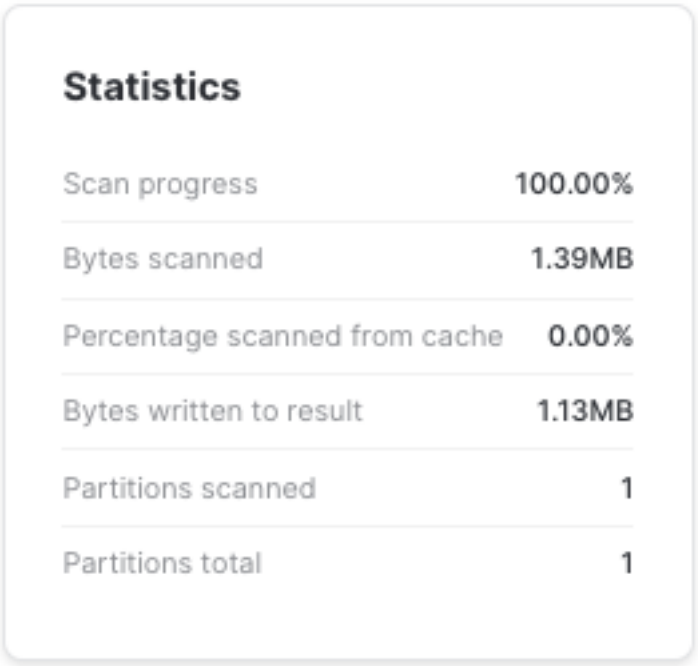


Figure 24: Statistics Panel

© 2023 Snowflake Computing Inc. All Rights Reserved

3.1.9 Click on Node A (TableScan[2]).

This node shows statistics related to the scan of the customer table. As we saw in a previous step, the Statistics panel shows that one partition was scanned out of one total partitions evaluated. The query was very simple, and the data set isn't very large, so this was to be expected.

Note that an Attributes panel appears that shows the columns selected during processing of this node's query.

© 2023 Snowflake Computing Inc. All Rights Reserved

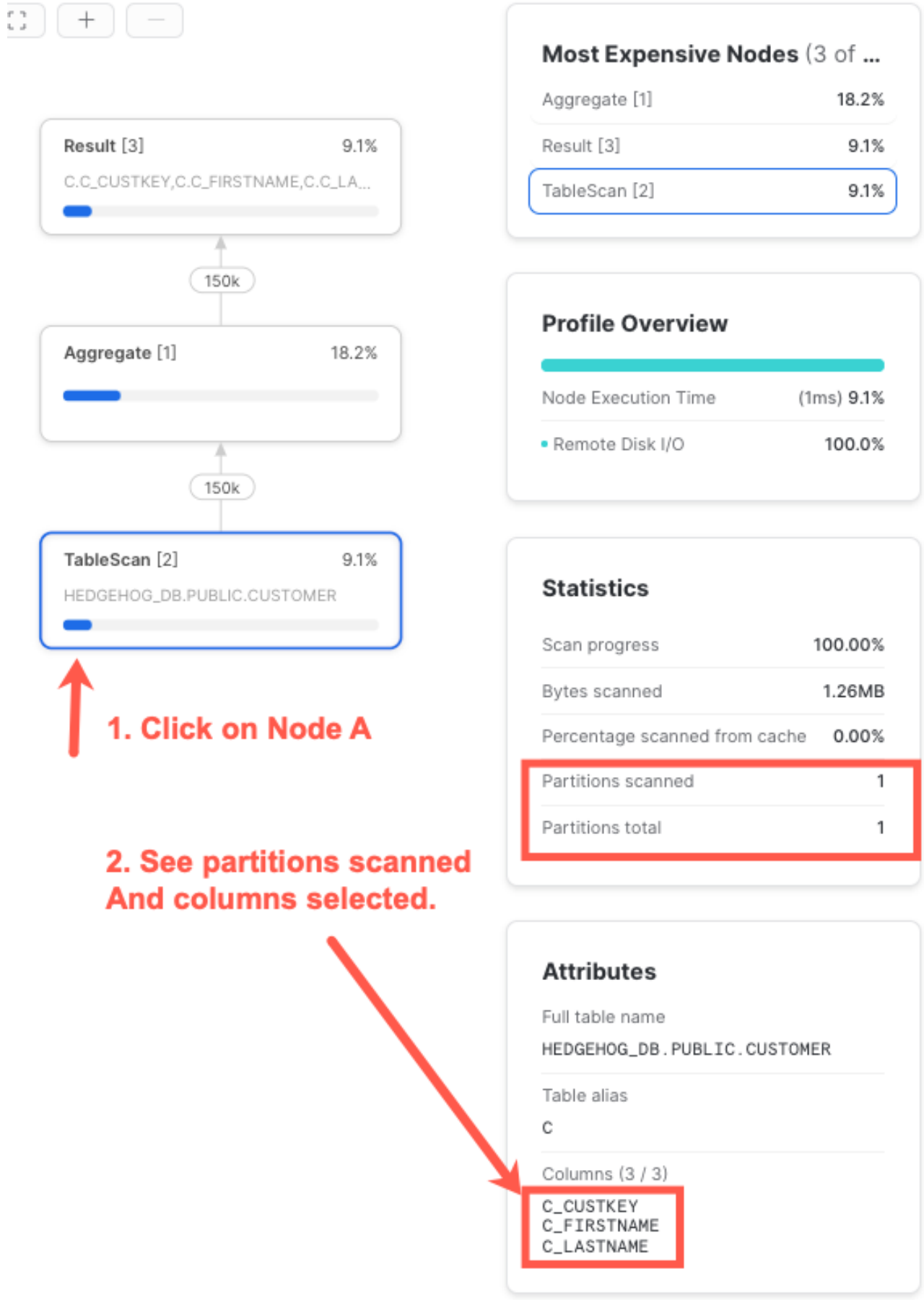


Figure 25: Node 1 Table Scan

© 2023 Snowflake Computing Inc. All Rights Reserved

3.1.10 Click on Node B (Aggregate[1]).

This node shows the statistics related to the aggregation of the data.

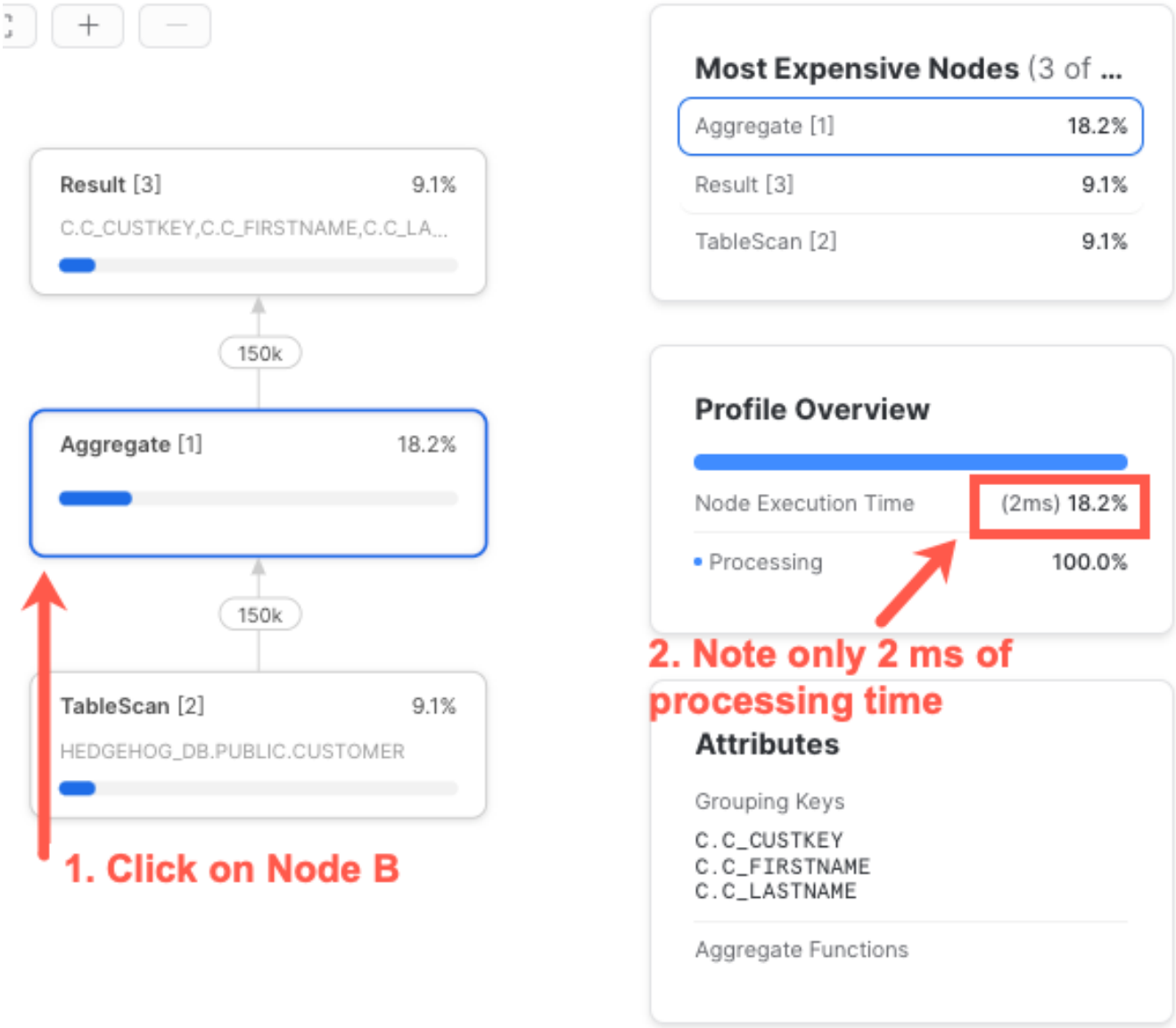


Figure 26: Node 2 - Aggregate

NOTE: You should also get similar results, but it's possible yours may differ slightly.

3.1.11 Click on Node C (Result[3]).

This node shows the columns that were in the output.

© 2023 Snowflake Computing Inc. All Rights Reserved

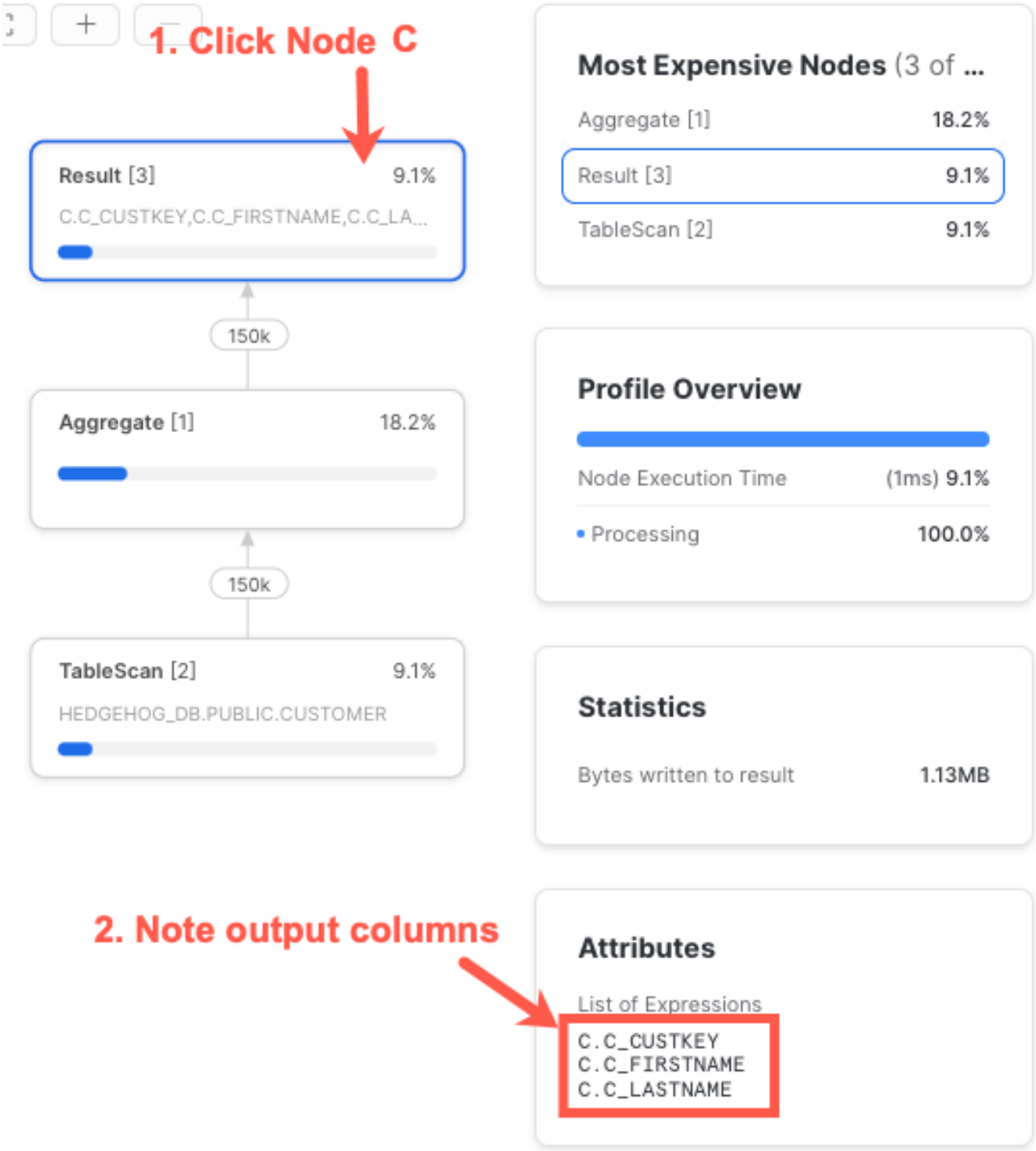


Figure 27: Node 3 - Result

Note that an Attributes panel appears that shows the columns produced by the processing of this node.

© 2023 Snowflake Computing Inc. All Rights Reserved

3.1.12 Close the tab.

You'll remember that the Query Profile opened up in a new tab. While we could navigate back to our folder, that would leave us with two tabs open. Close this tab and return to the tab with your worksheet.

3.1.13 Go back to the tab with your worksheet and run your query again.

Once you've run the query again, you should notice that it ran in a few milliseconds, much faster than before. Now let's look at the Query Profile again to see what happened.

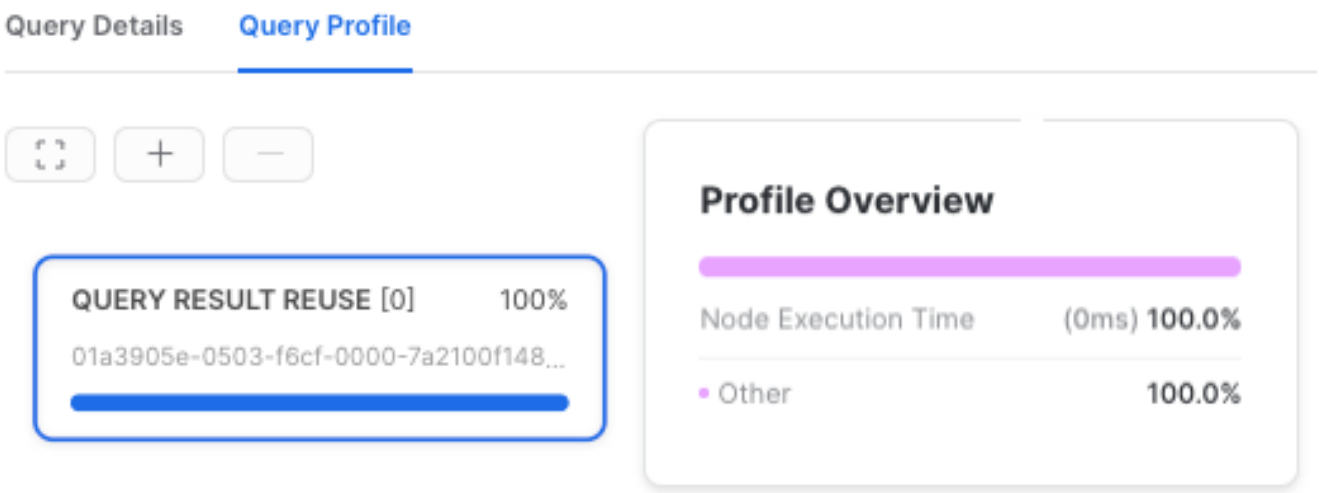


Figure 28: Query Result Cache

As you can see, the query gave us the exact same result in just a few milliseconds because it was serving us the same results from the query result cache in the cloud services layer. This is a ready-to-go feature that you don't have to think about. You can just run your query a second time and get the same results again if needed.

Close the Query Profile tab and return to your worksheet.

3.2 Metadata Cache

Metadata cache is simple. When data is written into Snowflake partitions, the MAX, MIN, COUNT, and other values are stored in the metadata cache in the Cloud Services layer. This means that when your query needs these values, rather than scanning the table and calculating the values, it simply pulls them from the metadata cache. This makes your query run much faster. Let's try it out!

3.2.1 Scenario.

Let's imagine you've been asked to analyze part and supplier data. We're going to use the PARTSUPP table in our database called SNOWFLAKE_SAMPLE_DATA because it provides enough data for this exercise.

© 2023 Snowflake Computing Inc. All Rights Reserved

3.2.2 Set the context.

Note we're setting USE_CACHED_RESULT = FALSE in order to avoid using the query result cache.

```
USE ROLE TRAINING_ROLE;
USE WAREHOUSE [login]_WH;
USE SCHEMA SNOWFLAKE_SAMPLE_DATA.TPCH_SF10;
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
```

3.2.3 Run the following SQL statement:

```
SELECT
          MIN(ps_partkey)
        , MAX(ps_partkey)

FROM
        PARTSUPP;
```

Now check the Query Profile. You should see a single node that says "METADATA-BASED RESULT". This is because the query profile simply went to cache to get the data you needed rather than scanning a table. So, there was no disk I/O at all.

Close the Query Profile tab and return to your worksheet.

3.3 Data Cache

Snowflake caches data from queries you run so it can be accessed later by other queries. This cache is saved to disk in the virtual warehouse. Let's take a look at how it works. Once again, let's assume you've been asked to analyze part and supplier data.

3.3.1 Clear your cache.

With the first statement below, you will ensure the query result cache is disabled so that you can focus on the data cache. With the remaining statements, you'll suspend and restart your virtual warehouse to ensure you're not using previously cached data.

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
ALTER WAREHOUSE [login]_WH SUSPEND;
ALTER WAREHOUSE [login]_WH RESUME;
```

3.3.2 Run the SQL statement below.

Let's start by selecting two columns, ps_suppkey and ps_availqty, with a WHERE clause that selects only part of the dataset. For any rows the query retrieves, this will cache the data for the two columns plus the column in the WHERE clause.

© 2023 Snowflake Computing Inc. All Rights Reserved

```
SELECT
          ps_partkey
        , ps_availqty

FROM
        PARTSUPP

WHERE
        ps_partkey > 1000000;
```

3.3.3 Look at percentage scanned from cache under Statistics in the Query Profile.

You should see that the percentage scanned from cache is 0.00%. This is because we just ran the query for the first time on a newly resumed virtual warehouse.

3.3.4 Run the query again.

```
SELECT
          ps_partkey
        , ps_availqty

FROM
        PARTSUPP

WHERE
        ps_partkey > 1000000;
```

3.3.5 Look at percentage scanned from cache under Statistics in the Query Profile.

You should see that the percentage scanned from cache is 100.00%. This is because the query was able to get 100% of the data it needed from the data cache. This results in faster performance than a query that does a lot of disk I/O.

3.3.6 Add columns, run the query, and check the Query Profile.

```
SELECT
          ps_partkey
        , ps_suppkey
        , ps_availqty
        , PS_supplycost
        , ps_comment

FROM
        PARTSUPP

WHERE
        ps_partkey > 1000000;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

When you check the percentage scanned from cache, it should be less than 100%. This is because we added columns that weren't fetched previously, so some disk I/O must occur in order to fetch the data in those columns.

Close the Query Profile tabs and return to your worksheet.

3.4 Partition Pruning

Partition pruning is a process by which Snowflake eliminates partitions from a table scan based on the partition's metadata. What this means is that fewer partitions are read from the storage layer or are involved in filtering and joining, which gives you faster performance.

Data in Snowflake tables will be organized based on how the data is ingested. For example, if the data in a table has been organized based on a particular column, knowing which column that is and including it in joins or in WHERE clause predicates will result in more partitions being pruned and, thus, faster query performance.

This organization is called natural clustering. As the details related to clustering, in general, are not within the scope of this course, you can learn more by checking our documentation or by taking the Snowflake Advanced course.

Let's look at an example. We're going to be using a different and larger dataset than our PROMO_CATALOG_SALES dataset so we can leverage partition pruning. Let's set the context, set our virtual warehouse size to xsmall, and clear our data cache.

3.4.1 Set the context, virtual warehouse size, and clear your cache.

```
USE ROLE TRAINING_ROLE;
USE WAREHOUSE [login]_WH;
USE SCHEMA SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL;
ALTER WAREHOUSE [login]_WH SET WAREHOUSE_SIZE = 'XSMALL';
ALTER SESSION SET USE_CACHED_RESULT=FALSE;
ALTER WAREHOUSE [login]_WH SUSPEND;
ALTER WAREHOUSE [login]_WH RESUME;
```

Remember, if your virtual warehouse is already suspended, either by timeout or command, you will receive the following error message upon execution of your SUSPEND statement: "Invalid state. Warehouse [login]_WH cannot be suspended".

3.4.2 Execute a query with partition pruning.

Let's imagine that the Snowbear Air marketing team has asked you for a list of customer addresses via a join on the CUSTOMER and CUSTOMER_ADDRESS tables. The data in the CUSTOMER table has been organized based on C_CUSTOMER_SK, which is a unique identifier for each customer. The WHERE clause filters on both C_CUSTOMER_SK and on C_LAST_NAME. Execute the query below and check the Query Profile to see what happens.

© 2023 Snowflake Computing Inc. All Rights Reserved

```
SELECT
          C_CUSTOMER_SK
        , C_LAST_NAME
        , (CA_STREET_NUMBER || ' ' || CA_STREET_NAME) AS CUST_ADDRESS
        , CA_CITY
        , CA_STATE
FROM
          CUSTOMER
          INNER JOIN CUSTOMER_ADDRESS ON C_CUSTOMER_ID = CA_ADDRESS_ID
WHERE
        C_CUSTOMER_SK between 100000 and 600000
        AND
        C_LAST_NAME LIKE 'Johnson'
ORDER BY
          CA_CITY
        , CA_STATE;
```

If you check the nodes for each table, you'll see that the CUSTOMER and CUSTOMER_ADDRESS tables have just over 500 total partitions between them.

The Query Profile tells us that the query ran in a few seconds, and only about half of the partitions were scanned. So this query ran faster than it would have otherwise because partition pruning worked for us.

Now let's run a query without the C_CUSTOMER_SK field in the WHERE clause predicate and see what happens.

3.4.3 Execute a query without partition pruning and check the Query Profile.

```
SELECT
          C_CUSTOMER_SK
        , C_LAST_NAME
        , (CA_STREET_NUMBER || ' ' || CA_STREET_NAME) AS CUST_ADDRESS
        , CA_CITY
        , CA_STATE
FROM
          CUSTOMER
          INNER JOIN CUSTOMER_ADDRESS ON C_CUSTOMER_ID = CA_ADDRESS_ID
WHERE
        C_LAST_NAME = 'Johnson'
ORDER BY
          CA_CITY
        , CA_STATE;
```

The Query Profile tells us that this query took longer to run, and all partitions were scanned. This is because the data in the CUSTOMER table is not organized on the C_LAST_NAME column, so more partitions had to be scanned in order for us to get our query result. The takeaway here is that understanding how your table's data is organized can help you write more efficient queries. Your DBA (Snowflake Administrator) is someone who can help you find that information.

© 2023 Snowflake Computing Inc. All Rights Reserved

3.5 Determine if Spillage is Taking Place

Now let's determine if spillage is taking place in one of our queries. Spillage means that because an operation cannot fit completely in memory, data is spilled to disk within the virtual warehouse. This is known as local spillage. When the disk in the virtual warehouse is full, the operation begins to spill to storage. This is known as remote spillage. Operations that incur spillage are slower than memory access and can greatly slow down query execution. Thus, you need to be able to identify and rectify spillage.

Let's imagine that Snowbear Air wants to determine the average list price, average sales price, and average quantity for both male and female buyers in the year 2000 for the months of January through October.

Rather than use our PROMO_CATALOG_SALES database for this scenario, we're going to use another database that has enough data to create spillage. The structure and content of the data is less important than the fact that we can generate and resolve a spillage issue.

3.5.1 Set the context and resize the virtual warehouse.

```
USE ROLE TRAINING_ROLE;
USE WAREHOUSE [login]_WH;
USE SCHEMA SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL;
ALTER WAREHOUSE [login]_WH SET WAREHOUSE_SIZE = 'XSMALL';
```

3.5.2 Clear the cache.

Run the SQL statements below to set USE_CACHED_RESULT to false, and suspend and resume your virtual warehouse to clear any cache. This will ensure we get spillage when we run our query.

```
ALTER SESSION SET USE_CACHED_RESULT=FALSE;
ALTER WAREHOUSE [login]_WH SUSPEND;
ALTER WAREHOUSE [login]_WH RESUME;
```

3.5.3 Run a query that generates spillage.

Note that the query below has a nested query. The nested query determines the average list price, average sales price, and average quantity per gender type and order number. The outer query then aggregates those values by gender.

Run the query below. It should take around 3 to 5 minutes to run (your results may vary).

```
SELECT
          cd_gender
        , AVG(lp) average_list_price
        , AVG(sp) average_sales_price
        , AVG(qu) average_quantity
FROM
        (
          SELECT
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
                  cd_gender
                , cs_order_number
                , AVG(cs_list_price) lp
                , AVG(cs_sales_price) sp
                , AVG(cs_quantity) qu
            FROM
                  catalog_sales
                , date_dim
                , customer_demographics
            WHERE
                  cs_sold_date_sk = d_date_sk
                  AND
                  cs_bill_cdemo_sk = cd_demo_sk
                  AND
                  d_year IN (2000)
                  AND
                  d_moy IN (1,2,3,4,5,6,7,8,9,10)
            GROUP BY
                  cd_gender
                , cs_order_number
          ) inner_query
GROUP BY
          cd_gender;
```

3.5.4 View the results.

For female buyers, you should see something very similar to the following figures:

- average_list_price: 100.995691505527
- average_sales_price: 50.494185840967
- average_quantity: 50.497579311044

For male buyers, you should see something very similar to the following figures:

- average_list_price: 100.992076976143
- average_sales_price: 50.491772005658
- average_quantity: 50.499846880459

3.5.5 Check out the Query Profile.

Go to the Query Profile. As you will see at the bottom of the Statistics, gigabytes of data were spilled to local storage. Notice there are two Aggregate nodes in the query. Click on each and notice the first Aggregate node is where the spillage is happening. This node is part of the inner query. Let's rectify this issue by rewriting our query.

If you look back at the query you just ran, you'll see that the outer query is not really necessary. All you need to do is remove the cs_order_number column from the nested query and then run it.

© 2023 Snowflake Computing Inc. All Rights Reserved

3.5.6 Run the modified nested query.

Let's run the query. We'll suspend the virtual warehouse first to flush any cache so we can get a true reading of how long it will take for the query to run.

```
ALTER WAREHOUSE [login]_WH SUSPEND;
ALTER WAREHOUSE [login]_WH RESUME;

SELECT
         cd_gender
       , AVG(cs_list_price) lp
       , AVG(cs_sales_price) sp
       , AVG(cs_quantity) qu
FROM
        catalog_sales
      , date_dim
      , customer_demographics
WHERE
      cs_sold_date_sk = d_date_sk
      AND
      cs_bill_cdemo_sk = cd_demo_sk
      AND
      d_year IN (2000)
      AND
      d_moy IN (1,2,3,4,5,6,7,8,9,10)
GROUP BY
          cd_gender;
```

3.5.7 Check your results.

The query should have run in 1-3 minutes (your results may vary). Compare your results to the ones you got previously. They may be slightly different past the hundreds place to the right of the decimal, but that is due to the differences in rounding between the original query and the modified nested query. So, in essence, you got the same results only in far less time.

3.5.8 Check the Query Profile.

As you will see at the bottom of the Statistics, there is no longer a spillage entry. This means that you resolved your spillage issue by simply rewriting your query so that it was more efficient.

3.6 Review the EXPLAIN Plan

Now let's compare the EXPLAIN plans from both of the queries we just ran in order to see how they are different.

3.6.1 Use EXPLAIN to see the plan for the first query.

© 2023 Snowflake Computing Inc. All Rights Reserved

```
EXPLAIN
SELECT
          cd_gender
        , AVG(lp) average_list_price
        , AVG(sp) average_sales_price
        , AVG(qu) average_quantity
FROM
        (
          SELECT
                  cd_gender
                , cs_order_number
                , AVG(cs_list_price) lp
                , AVG(cs_sales_price) sp
                , AVG(cs_quantity) qu
          FROM
                  catalog_sales
                , date_dim
                , customer_demographics
          WHERE
                cs_sold_date_sk = d_date_sk
                AND
                cs_bill_cdemo_sk = cd_demo_sk
                AND
                d_year IN (2000)
                AND
                d_moy IN (1,2,3,4,5,6,7,8,9,10)

          GROUP BY
                  cd_gender
                , cs_order_number
        ) inner_query
GROUP BY
        cd_gender;
```

3.6.2 Click on the operation column header and select the up arrow to sort the rows alphabetically by operation type.

Note that there are 12 rows that correspond to the execution nodes that you would see in the Query Profile. Also, note that two of the rows are aggregate rows. The node below executes the averaging of the list price, sales price, and quantity:

```
  aggExprs: [SUM((SUM(CATALOG_SALES.CS_LIST_PRICE)) / (COUNT(CATALOG_SALES.CS_LIST_PRICE)))
, COUNT((SUM(CATALOG_SALES.CS_LIST_PRICE)) / (COUNT(CATALOG_SALES.CS_LIST_PRICE)))
, SUM((SUM(CATALOG_SALES.CS_SALES_PRICE)) / (COUNT(CATALOG_SALES.CS_SALES_PRICE)))
, COUNT((SUM(CATALOG_SALES.CS_SALES_PRICE)) / (COUNT(CATALOG_SALES.CS_SALES_PRICE)))
, SUM((SUM(CATALOG_SALES.CS_QUANTITY)) / (COUNT(CATALOG_SALES.CS_QUANTITY)))
, COUNT((SUM(CATALOG_SALES.CS_QUANTITY)) / (COUNT(CATALOG_SALES.CS_QUANTITY)))]
, groupKeys: [CUSTOMER_DEMOGRAPHICS.CD_GENDER]
```

Figure 29: Aggregate Row Expression

© 2023 Snowflake Computing Inc. All Rights Reserved

3.6.3 Run the EXPLAIN statement for the second query.

```
EXPLAIN
SELECT
         cd_gender
       , AVG(cs_list_price) lp
       , AVG(cs_sales_price) sp
       , AVG(cs_quantity) qu
FROM
        catalog_sales
      , date_dim
      , customer_demographics
WHERE
      cs_sold_date_sk = d_date_sk
      AND
      cs_bill_cdemo_sk = cd_demo_sk
      AND
      d_year IN (2000)
      AND
      d_moy IN (1,2,3,4,5,6,7,8,9,10)
GROUP BY
          cd_gender;
```

Notice now that this plan is identical to the first one except that there is one aggregate row fewer than in the previous explain plan (for a total of 11 rows). Specifically, the node shown in the previous step in this lab is the one that is gone because we removed the outer query. Making that change alone was enough to cut query time by more than half.

3.6.4 Change your virtual warehouse size to XSmall.

```
ALTER WAREHOUSE [login]_WH
    SET WAREHOUSE_SIZE = 'XSmall';
ALTER WAREHOUSE [login]_WH SUSPEND;
```

3.7 Summary

Writing efficient queries is an art that takes a solid understanding of how Snowflake caching and query pruning impact query performance. While it's impossible to show you every single scenario, you should know that getting proficient at using tools like the Query Profile and the EXPLAIN plan will help you better understand how caching impacts your query performance. This, in turn, will allow you to write better queries that achieve a shorter run time.

3.8 Key takeaways

- Snowflake employs caching to help you get the query results you want as fast as possible.

- Cache is turned on by default, and it works for you in the background without you having to do anything.

© 2023 Snowflake Computing Inc. All Rights Reserved

- The Query Profile is a useful tool for understanding how caching and partition pruning are impacting your queries.

- As you add or remove columns to/from a SELECT clause or a WHERE clause, your percentage scanned from cache value could go up or down.

- If your query only requests MIN or MAX values on INTEGER, DATE or DATETIME data types, those values will come from metadata cache in the Cloud Services layer rather than from disk I/O, which results in fast performance.

- Query Result cache is invoked when you run the exact same query twice.

- Data cache resides in the virtual warehouse, and it stores data from past queries on a least recently used (LRU) basis, until the virtual warehouse is suspended. However, once the virtual warehouse is suspended, its data cache is cleared out.

- Including the column on which a table's data is organized in a WHERE clause predicate can improve partition pruning, which in turn improves performance.

- Using EXPLAIN can give you insight into how Snowflake will execute your query. You can use it to identify and remove bottlenecks in your query so you can resolve them and get better efficiency.

© 2023 Snowflake Computing Inc. All Rights Reserved

4 Loading Structured Data

The purpose of this lab is to introduce you to data loading.

Learning Objectives:

- How to load data from a file in an external stage into a table using the COPY INTO command.
- How to define a GZIP file format.
- How to review the properties of a stage.
- How to load a GZipped file from an external stage into a table.
- How to validate data prior to loading
- How to handle data loading errors

Scenario:

You are a data engineer at Snowbear Air. You need to create and populate a table that will be used in reporting. The table will be named REGION, and you will populate it from a pre-existing file (`region.tbl`) in an external stage. The file is headerless, pipe-delimited, and contains five rows.

HOW TO COMPLETE THIS LAB

In order to complete this lab, you can type the SQL commands below directly into a worksheet. It is not recommended that you cut and paste from the workbook pdf as that sometimes results in errors.

You can also use the SQL code file for this lab that was provided at the start of the class. To open an .SQL file in Snowsight, make sure the Worksheet section is selected on the left navigation bar. Click on the ellipsis between the Search and +Worksheet buttons. In the dropdown menu, select “Create Worksheet from SQL File”.

Let’s get started!

4.1 Loading Data From an External Stage Into a Table Using COPY INTO

In this exercise, you will learn how to load a file from an external stage into a table using the COPY INTO command.

4.1.1 Create a new folder called Data Loading.

4.1.2 Create a new worksheet named *Load Structured Data.*

4.1.3 Use the following SQL to set the context:

```
USE ROLE TRAINING_ROLE;
CREATE WAREHOUSE IF NOT EXISTS [login]_WH;
USE WAREHOUSE [login]_WH;
CREATE DATABASE IF NOT EXISTS [login]_DB;
USE [login]_DB.PUBLIC;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

4.1.4 Create a REGION table. This table will be loaded from a source file:

```
CREATE OR REPLACE TABLE REGION (
       R_REGIONKEY NUMBER(38,0) NOT NULL,
       R_NAME      VARCHAR(25)  NOT NULL,
       R_COMMENT   VARCHAR(152)
);
```

4.1.5 Create a file format called `MYPIPEFORMAT`, that will read the pipe-delimited `region.tbl` file:

```
CREATE OR REPLACE FILE FORMAT MYPIPEFORMAT
  TYPE = CSV
  COMPRESSION = NONE
  FIELD_DELIMITER = '|'
  FILE_EXTENSION = 'tbl'
  ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE;
```

4.1.6 Create a file format called `MYGZIPPIPEFORMAT` that will read the compressed version of the region.tbl file. It should be identical to the `MYPIPEFORMAT`, except you will set `COMPRESSION = GZIP`.

```
CREATE OR REPLACE FILE FORMAT MYGZIPPIPEFORMAT
 TYPE = CSV
 COMPRESSION = GZIP
 FIELD_DELIMITER = '|'
 FILE_EXTENSION = 'tbl'
 ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE;
```

4.2 Load the *region.tbl* File

The files for this task have been pre-loaded into a location on AWS. The external stage that points to that location has been created for you. The stage is in the TRAININGLAB schema of the TRAINING_DB database. In this task, you will review the files in the stage and load them using the file formats you created.

4.2.1 Review the properties of the stage:

Take note of the field delimiter for the file format in the stage.

```
DESCRIBE STAGE TRAINING_DB.TRAININGLAB.ED_STAGE;
```

4.2.2 Review the properties of the file:

Take note of the delimiter used in the file.

```
SELECT n.$1
FROM @training_db.traininglab.ed_stage/load/lab_files/region.tbl n;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

NOTE: As you may have noticed, the field delimiter for the file format defined in the stage is set to a comma, not a pipe. However, the file data is in pipe format. You have two options to load the file despite this discrepancy. First, you can override the stage's file format by specifying a different file format to be used when the COPY command executes. Second, you can load the data in pipe format even though the stage uses a comma as a field delimiter. In this exercise, you will override the stage's file format by configuring the COPY statement to use the MYPIPEFORMAT file format instead when it executes.

4.2.3 Confirm the **region.tbl** file is in the external stage with the list command:

```
LIST @training_db.traininglab.ed_stage/load/lab_files/ pattern='.*region.*';
```

4.2.4 Load the data from the external stage to the REGION table using the file format you created in the previous task:

```
COPY INTO REGION
FROM @training_db.traininglab.ed_stage/load/lab_files/
FILES = ('region.tbl')
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT);
```

4.2.5 Select and review the data in the REGION table, either by executing the following command in your worksheet or by using **Preview Data** in the sidebar:

```
SELECT * FROM REGION;
```

4.3 Loading a GZip Compressed File on an External Stage Into a Table

The scenario for this activity is fundamentally the same as the previous activity. The difference is that you will load the REGION table from a gzip compressed file that is in the external stage. You will use the MYGZIPPIPEFORMAT file format you created in the previous part of this lab.

4.3.1 Empty the REGION table in the PUBLIC schema of **[login]**_DB:

```
TRUNCATE TABLE region;
```

4.3.2 Confirm that the `region.tbl.gz` file is in the external stage:

```
LIST @training_db.traininglab.ed_stage/load/lab_files/ pattern='.*region.*';
```

© 2023 Snowflake Computing Inc. All Rights Reserved

4.3.3 Reload the REGION table from the region.tbl.gz file. Review the syntax of the COPY INTO command used in the previous task. Specify the file to COPY as 'region.tbl.gz'.

```
COPY INTO region
FROM @training_db.traininglab.ed_stage/load/lab_files/
FILES = ('region.tbl.gz')
FILE_FORMAT = ( FORMAT_NAME = MYGZIPPIPEFORMAT);
```

4.3.4 Query the table to confirm the data was successfully loaded:

```
SELECT * FROM region;
```

4.3.5 Resize and suspend the virtual warehouse.

```
ALTER WAREHOUSE [login]_WH SET WAREHOUSE_SIZE=XSmall;
ALTER WAREHOUSE [login]_WH SUSPEND;
```

4.4 Validating Data Prior to Load

Now we're going to practice using the VALIDATION_MODE parameter of the COPY INTO statement to check for problems with the file prior to loading it.

NOTE: Some of the steps will intentionally produce errors. Read and follow the instructions closely, so you understand what is happening in each step.

4.4.1 Modify the NATION table by running the statement below.

```
CREATE OR REPLACE TABLE nation (
                          NATION_KEY INTEGER
                        , NATION VARCHAR
                        , REGION_KEY INTEGER
                        , COMMENTS VARCHAR
                    );
```

4.4.2 Try to copy into the NATION table by executing the COPY INTO statement below.

Notice that the validation mode is RETURN_ALL_ERRORS. This will return any and all errors if there are any. Regardless, no data will be loaded. This is because by providing a value for VALIDATION_MODE, we are indicating we only want to check the file, not load the file.

```
COPY INTO NATION
FROM @training_db.traininglab.ed_stage/load/lab_files/
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
FILES = ('nation.tbl')
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
VALIDATION_MODE = RETURN_ALL_ERRORS;
```

You should have gotten a message that says, “Query produced no results”. This means there were no errors and that you can load the table. But now we’re going to recreate the table and switch the order of the columns. By making REGION_KEY, which is an integer column, the second column in the table, we will have errors because the second column in the file is a VARCHAR field.

4.4.3 Recreate the NATION table and execute the COPY INTO statement.

Note that in this case, we are using RETURN_ERRORS. Like RETURN_ALL_ERRORS, it will return any and all errors stemming from the loading of the file indicated in the COPY INTO statement below.

```
CREATE OR REPLACE TABLE nation (
                          NATION_KEY INTEGER
                        , REGION_KEY INTEGER
                        , NATION VARCHAR
                        , COMMENTS VARCHAR
                    );

COPY INTO NATION
FROM @training_db.traininglab.ed_stage/load/lab_files/
FILES = ('nation.tbl')
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
VALIDATION_MODE = RETURN_ERRORS;
```

Now you should have 25 rows indicating that the VARCHAR value we tried to load into the REGION column has created an error. Had we tried to load the file, none of the rows would have loaded.

4.4.4 Check only the first ten rows.

In the next statement, we set the VALIDATION_MODE to RETURN_10_ROWS. So, our statement will only check the first ten rows and return the first error it encounters.

```
COPY INTO NATION
FROM @training_db.traininglab.ed_stage/load/lab_files/
FILES = ('nation.tbl')
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
VALIDATION_MODE = RETURN_10_ROWS;
```

As you can see, we have a message saying that the numeric value “ALGERIA” is not recognized.

4.5 Error Handling

Now we’re going to examine error handling options for data loading in Snowflake.

© 2023 Snowflake Computing Inc. All Rights Reserved

4.5.1 Recreate the table with all columns in the same order as they are in the nation.tbl file.

```
CREATE OR REPLACE TABLE nation (
                          NATION_KEY INTEGER
                        , NATION VARCHAR
                        , REGION_KEY INTEGER
                        , COMMENTS VARCHAR
                    );
```

4.5.2 Verify the query.

As you will see in the query below, we've introduced an error in the third column. For all rows where the region key is 1, we've converted it to the VARCHAR value "America". This will generate five errors when loading into an INTEGER column.

```
SELECT
          n.$1 AS N_KEY
        , n.$2 AS NATION
        , CASE
            WHEN n.$3 = 1 THEN 'AMERICA'
            ELSE n.$3
          END AS R_KEY
        , n.$4 AS COMMENTS
FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl (file_format =>
   'MYPIPEFORMAT') n;
```

4.5.3 Attempt to load the data.

Notice that we've set the ON_ERROR parameter to continue. This means that all rows that don't generate an error will get loaded.

```
COPY INTO nation
FROM (
        SELECT
              n.$1 AS N_KEY
            , n.$2 AS NATION
            , CASE
                WHEN n.$3 = 1 THEN 'AMERICA'
                ELSE n.$3
              END AS R_KEY
            , n.$4 AS COMMENTS

        FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl
            (file_format => 'MYPIPEFORMAT') n
    )
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
ON_ERROR = CONTINUE;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

As you can see from the results, the status is PARTIALLY_LOADED and rows_loaded is 20 out of the original 25.

4.5.4 Run the SELECT statement below to verify the contents of the table, then truncate the table.

```
SELECT * FROM nation;

TRUNCATE TABLE nation;
```

4.5.5 Retry the insert with ON_ERROR = ABORT_STATEMENT.

ABORT_STATEMENT is the default value, and it will cause the entire load to fail.

```
COPY INTO nation
FROM (
        SELECT
              n.$1 AS N_KEY
            , n.$2 AS NATION
            , CASE
                WHEN n.$3 = 1 THEN 'AMERICA'
                ELSE n.$3
              END AS R_KEY
            , n.$4 AS COMMENTS

        FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl
           (file_format => 'MYPIPEFORMAT') n
    )
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
ON_ERROR = ABORT_STATEMENT; --ALSO THE DEFAULT OPTION WHEN ON_ERROR ISN'T SPECIFIED
```

As you can see, the error is “Numeric value ‘AMERICA’ is not recognized”. No data was loaded.

4.5.6 Retry the insert with ON_ERROR = SKIP_FILE_4.

With this statement, we’re telling Snowflake that we don’t want to load the file if we have four or more errors. Since we know our file will generate five errors, the load should fail completely.

```
COPY INTO nation
FROM (
        SELECT
              n.$1 AS N_KEY
            , n.$2 AS NATION
            , CASE
                WHEN n.$3 = 1 THEN 'AMERICA'
                ELSE n.$3
              END AS R_KEY
            , n.$4 AS COMMENTS
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
        FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl
           (file_format => 'MYPIPEFORMAT') n
    )
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
ON_ERROR = SKIP_FILE_4;
```

As you can see, you have a status that shows “LOAD_FAILED”.

4.5.7 Run query to validate error.

Run the query below to validate that the table is empty.

```
SELECT * FROM nation;
```

4.5.8 Retry the insert with ON_ERROR = SKIP_FILE_6.

With this statement, we’re telling Snowflake that we don’t want to load the file if we have six or more errors. Since we know our file will generate five errors, we should get a partial load.

```
COPY INTO nation
FROM (
        SELECT
              n.$1 AS N_KEY
            , n.$2 AS NATION
            , CASE
                WHEN n.$3 = 1 THEN 'AMERICA'
                ELSE n.$3
              END AS R_KEY
            , n.$4 AS COMMENTS

        FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl
           (file_format => 'MYPIPEFORMAT') n
    )
FILE_FORMAT = (FORMAT_NAME = MYPIPEFORMAT)
ON_ERROR = SKIP_FILE_6;
```

As you can see, the file was partially loaded. 20 out of 25 rows were loaded.

4.6 Key Takeaways

- Files to be loaded can be compressed or not compressed.

- The COPY INTO command can be used to load.

- You can use the LIST command to see what files are in a stage.

- Use the VALIDATION_MODE parameter of the COPY INTO statement to check for problems with the file prior to loading it.

© 2023 Snowflake Computing Inc. All Rights Reserved

- You can set the ON_ERROR parameter of the COPY INTO statement to load all, some, or none of the data if one or more errors are detected.

© 2023 Snowflake Computing Inc. All Rights Reserved

5 Data Transformation During Data Loading

The purpose of this lab is to introduce you to how to transform data when loading it.

Learning Objectives:

- How to transform data during the loading process

Scenario:

You are a data engineer at Snowbear Air. You need to populate a table with a list of nations, and you need to provide a region name as well as a region code. However, the file doesn't have those two pieces of data. You will need to transform the data in the file prior to loading it, so it has that data.

HOW TO COMPLETE THIS LAB

In order to complete this lab, you can type the SQL commands below directly into a worksheet. It is not recommended that you cut and paste from the workbook pdf as that sometimes results in errors.

You can also use the SQL code file for this lab that was provided at the start of the class. To open an .SQL file in Snowsight, make sure the Worksheet section is selected on the left navigation bar. Click on the ellipsis between the Search and +Worksheet buttons. In the dropdown menu, select "Create Worksheet from SQL File".

Let's get started!

5.1 Transforming Data During Load

5.1.1 Create a new folder called Data Loading and Transformation.

5.1.2 Create a new worksheet named *Transforming Data During Load.*

5.1.3 Create objects and set the context.

```
CREATE DATABASE IF NOT EXISTS [login]_db;
USE DATABASE [login]_db;
CREATE SCHEMA IF NOT EXISTS [login]_transform;
USE SCHEMA [login]_db.[login]_transform;
```

5.1.4 Search for the nation file.

Execute the statement below to list the files in the stage with a .tbl extension.

```
LIST @training_db.traininglab.ed_stage/load/lab_files/ pattern='.*\.tbl.*';
```

If you scroll down far enough, you will find nation.tbl. That is the file that you want to transform and load.

© 2023 Snowflake Computing Inc. All Rights Reserved

5.1.5 Query the file in the stage.

Before you run the statement below, take a close look at the SELECT clause. Notice that there are three columns that start with the alias for the file (n) and are followed by a period, a dollar sign, and a number. These refer to the first three columns in the file. You can refer to columns in the SELECT clause in this way, whether or not they exist in the file. If they don't, the entire column will simply be null.

```
SELECT n.$1, n.$2, n.$3
FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl n ;
```

Note that the first two columns have data, while the third one is null. In fact, the first column is the one we want. The problem is that we want to transform the data prior to loading it into a table, and it's in a pipe-delimited format. So, we need to create a file format that will handle the data and let us query individual columns straight out of the file.

5.1.6 Create a pipe-delimited file format.

```
CREATE OR REPLACE FILE FORMAT MYPIPEFORMAT
  TYPE = CSV
  COMPRESSION = NONE
  FIELD_DELIMITER = '|'
  FILE_EXTENSION = 'tbl'
  ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE;
```

5.1.7 Create the target table.

We've been given a target table format. Let's create the table now:

```
CREATE OR REPLACE TABLE nation (
                                  NATION_KEY INTEGER
                                , REGION VARCHAR
                                , REGION_CODE VARCHAR
                                , NATION VARCHAR
                                , COMMENTS VARCHAR
                             );
```

5.1.8 Use the file format to query the file.

As you can see, we've added the file format right after the file name. We've also aliased all of the columns.

```
SELECT n.$1 AS N_KEY, n.$2 AS N_NAME, n.$3 AS R_KEY, n.$4 AS N_COMMENT
FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl (file_format =>
   'MYPIPEFORMAT') n ;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

N_KEY can be the source for NATION_KEY in our target table. However, we only have a region key, not a region name. Also, we need to provide a region code that consists of the first two letters of the region name. Finally, N_NAME can be the source for NATION. The problem is that it is the second column in the file, but it needs to go into the fourth column in the table.

Fortunately, Snowflake allows us to insert columns in a different order than they exist in the file. Also, we can use functions and CASE…WHEN statements to transform the data into the format we need it to be in. The query below does that for us.

5.1.9 Execute the statement below.

```
SELECT    n.$1 AS N_KEY
        , CASE
            WHEN n.$3 = 0 THEN 'AFRICA'
            WHEN n.$3 = 1 THEN 'AMERICA'
            WHEN n.$3 = 2 then 'ASIA'
            WHEN n.$3 = 3 THEN 'EUROPE'
            ELSE 'MIDDLE EAST'
          END AS REGION
        , substr(REGION, 1, 2) AS REGION_CODE
        , n.$2 AS NATION
        , n.$4 AS COMMENTS

FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl (file_format =>
   'MYPIPEFORMAT') n ;
```

As you can see, the columns are now in the order and format that we need. Let's load the data now.

5.1.10 Load the data.

Execute the COPY INTO statement below and then check the results.

```
COPY INTO nation
FROM (
        SELECT
              n.$1 AS N_KEY
            , CASE
                WHEN n.$3 = 0 THEN 'AFRICA'
                WHEN n.$3 = 1 THEN 'AMERICA'
                WHEN n.$3 = 2 then 'ASIA'
                WHEN n.$3 = 3 THEN 'EUROPE'
                ELSE 'MIDDLE EAST'
              END AS REGION
            , substr(REGION, 1, 2) AS REGION_CODE
            , n.$2 AS NATION
            , n.$4 AS COMMENTS

        FROM @training_db.traininglab.ed_stage/load/lab_files/nation.tbl
           (file_format => 'MYPIPEFORMAT') n
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
    );
```

In the result pane, you should see a row that says status = “LOADED” and rows_loaded = “25”. If you do, the load was successful.

5.1.11 Run the SELECT statement below to view the contents of the NATION table.

```
SELECT * FROM nation;
```

You should have 25 rows of data in your table.

5.1.12 Suspend and resize the virtual warehouse.

```
ALTER WAREHOUSE [login]_WH SET WAREHOUSE_SIZE=XSmall;
ALTER WAREHOUSE [login]_WH SUSPEND;
```

5.2 Key Takeaways

- You can transform data prior to loading it.

- When transforming data, you can reorder the columns coming out of the file, and you can use (but are not limited to) functions or CASE…WHEN statements.

© 2023 Snowflake Computing Inc. All Rights Reserved

6 Exploring Semi-Structured JSON Data

In this lab, you will create a simple table that includes a VARIANT column. You will work with that simple table before you go on to explore more complex data using both dot notation syntax and the `FLATTEN` function.

Learning Objectives:

By the end of this lab, you will be able to:
- Create a simple table with JSON data
- Query a column containing JSON data using dot notation syntax
- Query a column containing JSON data using the `FLATTEN` function
- Determine the data types of a query result

Scenario:

Snowbear Air has started to collect some data in JSON format, but none of the analysts are familiar with using semi-structured data. The management team at Snowbear Air has tasked you with learning about this new data type, and teaching the other analysts what you have learned. You decide to begin by creating, and then working with, some simple JSON data.

HOW TO COMPLETE THIS LAB

As the workbook PDF may have useful diagrams, we recommend that you read the instructions from the workbook PDF. In order to execute the code presented in each step, use the SQL code file that was provided for this lab.

OPENING THE SQL FILE

To load the SQL file, select **Worksheets** in the left navigation bar and click the ellipses in the upper-right corner of your Snowsight window. Select **Create Worksheet from SQL File** from the drop-down menu. Navigate to the lab file for this lab and load it.

Let's get started!

6.1 Query Simple JSON Data Using dot notation syntax

6.1.1 Make sure you have the required objects, and set your context for this lab:

```
USE ROLE training_role;

CREATE WAREHOUSE IF NOT EXISTS [login]_WH;
ALTER WAREHOUSE [login]_WH SET WAREHOUSE_SIZE=XSmall;
USE WAREHOUSE [login]_WH;

CREATE DATABASE IF NOT EXISTS [login]_DB;
USE DATABASE [login]_DB;

CREATE SCHEMA IF NOT EXISTS [login]_SCHEMA;
USE SCHEMA [login]_SCHEMA;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

6.1.2 Create a simple JSON file.

Use the SQL provided to create a simple table with two columns. The first column is a customer ID number, and the second column contains a JSON record with information about the customer. It uses the `parse_json` function to create the JSON data. It also uses the `$1` notation to refer to the first column in the table, and `$2` to refer to the second column.

```
CREATE OR REPLACE TABLE customers AS
SELECT
   $1 AS id,
   parse_json($2) AS info
FROM
   VALUES
      (12712555, '{"name": {"first": "John", "last":"Smith"}}'),
      (98127771, '{"name": {"first": "Jane", "last":"Doe"}}');
```

6.1.3 Open a row of the table you just created to see its structure.

```
SELECT * FROM customers;
```

Click on a row in the `info` column to display its structure on the right. Notice that the top-level key is called `name`, and its value is an object (a collection of key-value pairs).

6.1.4 Extract the name data from your table using dot notation syntax.

One way to extract values from JSON data is with dot notation syntax.

> **Dot Notation Syntax:**
> In your SELECT statement, you would simply reference the column, add a colon, then reference the elements one-by-one separated by dots:
> <column>:<level1_element>.<level2_element>.<level3_element>

Run the SELECT statement below to see dot notation syntax at work:

```
SELECT
   id,
   info:name.first AS first_name,
   info:name.last AS last_name
FROM
   customers;
```

For first_name, we reference column **info**, then follow it with a colon, then level 1 element **name**, then level 2 element **first**. We do something similar for last_name.

© 2023 Snowflake Computing Inc. All Rights Reserved

6.1.5 Determine the data types in your result set.

Below we'll use the command DESCRIBE RESULT and the function last_query_id() to fetch details about the query we just ran.

```
DESCRIBE RESULT last_query_id();
```

Notice that the name columns are still of data type `VARIANT`. Let's look at another way to determine if our data is of data type `VARIANT`.

6.1.6 Rerun the query examine the output

```
SELECT
   id,
   info:name.first AS first_name,
   info:name.last AS last_name
FROM
   customers;
```

You'll notice that the `first_name` and `last_name` column values are enclosed in double-quotes. Seeing string values in double-quotes is an indication that the output is probably still a `VARIANT`.

While you can use VARIANT data types in comparisons or aggregations with other data types, your queries won't be as performant as they would be if you had cast the VARIANT columns to standard SQL data types.

Let's observe how we can convert VARIANTS to a standard SQL data type.

6.1.7 Extract the name data and cast it to type VARCHAR.

```
SELECT
   id,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name
FROM
   customers;

DESCRIBE RESULT last_query_id();
```

As you can see, your first_name and last_name columns are now VARCHAR values. This allows you to sort them, apply string functions to them, or do anything you could with structured data.

6.1.8 Create a table from your query

```
CREATE OR REPLACE TABLE customers_structured
AS
   SELECT
      id,
      info:name.first::VARCHAR AS first_name,
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
      info:name.last::VARCHAR AS last_name
   FROM
      customers;

SELECT * FROM customers_structured;
```

You now have a table that can be used for analysis by your colleagues who do not know how to directly query semi-structured data.

6.2 Query Nested JSON Data with Dot Notation

There are two ways to query nested JSON data: one is with dot notation, and the other is with the FLATTEN function. Let's look at dot notation first.

6.2.1 Create a new version of your `customers` table with various levels of nested JSON data

The table below has various levels of nested values. As you can see, `contact` has both `business` and `personal` details. Key-value pairs `phone` and `email` are nested within both `business` and `personal`.

Run the code below to create the table.

```
CREATE OR REPLACE TABLE customers
AS
SELECT
   $1 AS id,
   parse_json($2) AS info
FROM VALUES
   (
   12712555,
   '{"name": {"first":"John", "last":"Smith"},
     "contact": [
        {"business": {"phone":"303-555-1234", "email":"j.smith@company.com"}},
        {"personal": {"phone":"303-421-8322", "email":"jsmith332@gmail.com"}}
        ],
     }'
   ),
   (
   98127771,
   '{"name": {"first":"Jane", "last":"Doe"},
     "contact": [
        {"business": {"phone":"303-638-4887", "email":"jg_doe@company2.com"}},
        {"personal": {"phone":"303-678-6789", "email":"happyjane@gmail.com"}}
        ],
     }'
   );
```

Notice that the value for key `contact` is enclosed in square brackets. This is because the value is an array of key value pairs, each of which contains two key-value pairs. Whenever you see those square brackets, you will need to do something extra with your dot notation in order to fetch the values you want.

© 2023 Snowflake Computing Inc. All Rights Reserved

6.2.2 Select from your table and click a cell in the INFO column to view the structure.

```
SELECT * FROM customers;
```

Notice that there are two people: John Smith and Jane Doe. Each has a key-value called `contact`, with `business` and `personal` contact details nested within. Let's query the table to fetch both people and their business and personal phone numbers using the same dot notation syntax we've been using.

6.2.3 Query the table to fetch business and personal phone numbers

```
SELECT
   ID,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name,
   info:contact.business.phone::VARCHAR AS business_phone,
   info:contact.personal.phone::VARCHAR AS personal_phone
FROM
   customers;
```

Notice that while our query fetched the full names of both John and Jane, the business phone and personal phone columns are null.

The key `contact` has a value consisting of a zero-based array of elements, each of which is a key-value pair. Because the keys `business` and `personal`, along with their values are nested within an array, we can't simply query `info:contact.business` and expect to get the business phone number. That's because we haven't told the query engine which element in the array we want, the first or the second.

Thus, in order to access the value of `business` (the first element in the array) we have to give the query engine an integer indicating a position in the array (an `index`). In our example the positions are 0 for `business` and 1 for `personal`. Remember, the first position is 0 because the array is zero-based.

Run the SELECT statement below to see this in action.

```
SELECT
   ID,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name,
   info:contact[0].business.phone::VARCHAR AS business_phone,
   info:contact[1].personal.phone::VARCHAR AS personal_phone
FROM
   customers;
```

As you can see, we indicate `contact[0]` for the business phone number and `contact[1]` for the personal phone number. Thus, the query engine understood which path it needed to follow. It then followed the dot notation (`business.phone` and `personal.phone`) to fetch the values we wanted.

6.2.4 Fetch all data and put it in structured format:

© 2023 Snowflake Computing Inc. All Rights Reserved

```
CREATE OR REPLACE TABLE customers_dot
AS
SELECT
   ID,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name,
   info:contact[0].business.phone::VARCHAR AS business_phone,
   info:contact[0].business.email::VARCHAR AS business_email,
   info:contact[1].personal.phone::VARCHAR AS personal_phone,
   info:contact[1].personal.email::VARCHAR AS personal_email
FROM
   customers;

SELECT * FROM customers_dot;
```

As you can see, we've now converted our semi-structured data to a structured format so it can be used by colleagues that need to join it with other structured data, or that don't know how to work with semi-structured data.

6.3 Use `FLATTEN` to Query JSON Data

Now we're going to use the FLATTEN function (instead of dot notation) to produce a structured representation of the same semi-structured data we were working with earlier.

FLATTEN is a table function that explodes the contents of a variant column (for purposes of this lab, a column with semi-structured data) and produces a table-like representation of the data.

The nice thing about the FLATTEN function is that it gives you some additional output fields that you can use to figure out how your data is structured and to fine-tune your query to get the results that you want.

6.3.1 Use the `FLATTEN` function to view all the top-level keys from your table.

The top-level keys are the first level of key-value pairs that the query engine finds when parsing the semi-structured data indicated in your query.

Run the query below to find those keys.

```
SELECT
   *
FROM
   customers,
LATERAL FLATTEN(input=>info);
```

When we use the FLATTEN function, its output is in essence joined to table `customers` and its columns can be selected just as you would with columns from a table of structured data.

First, notice that the input of the FLATTEN function is `(input=\>\<column\>)`. For '<column>' you can use just the column name or a dot notation path like the ones we saw earlier in this lab.

© 2023 Snowflake Computing Inc. All Rights Reserved

Now notice that your output contains eight columns. The first two (`ID` and `INFO`) are columns from the `customers` table, and the last six are the output of the `FLATTEN` function. Look at the `key` column - this lists all of the top-level keys in the `INFO` column.

This might be a little easier to see if you just pull out the `key` and `value` columns:

```
SELECT
   key,
   value
FROM
   customers,
LATERAL FLATTEN(input=>info);
```

You get four rows of output: the first two rows are the top-level keys (`contact` and `name`) from John Smith's record, and the second two rows are the same keys from Jane Doe's record. But what if you want to see all the keys, even those nested in other structures?

6.3.2 Use the `RECURSIVE` option to view all the keys from your table.

This time you will include the `index`:

```
SELECT
   key, index, value
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true);
```

Note that you have gone from four rows of output to 24. When you include the `RECURSIVE` option, any time a value is either an object or an array (as opposed to a simple value), then that object or array is also flattened.

Note that when the `index` column shows 0, the `value` column shows the business details. When the `index` column shows 1, the `value` column shows the personal details. This is consistent with what we saw in the earlier dot notation exercise.

6.3.3 Show only the keys that have simple values.

When you query semi-structured data, you are typically accessing keys that have simple values. For example, you're looking for the business phone, rather than the entire array of contact information. You can do this with a `WHERE` clause that uses the function `TYPEOF` to check the type of value being returned.

In the query below, you will check the contents of the `value` column from the `FLATTEN`.

```
SELECT
   key, index, value
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true)
WHERE TYPEOF(value) NOT IN ('OBJECT', 'ARRAY');
```

© 2023 Snowflake Computing Inc. All Rights Reserved

Notice that the `index` column contains nothing but null values. This is because the `index` column tells you the position of values in an array - but you filtered out all of the arrays with your WHERE clause. So there's no point in including the `index` column in the query.

```
SELECT
   key, value
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true)
WHERE TYPEOF(value) NOT IN ('OBJECT', 'ARRAY');
```

Now you know all the simple key:value pairs in each row, but you may still have some questions about the data. For example, several of the rows have a key of `phone` - but is that a business phone, or a personal phone? And is it for John or for Jane? The columns **`this`**, `seq`, and `path` from the `FLATTEN` function can help answer these questions.

6.3.4 Include the column `this` in your query.

```
SELECT
   key, value, this
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true)
WHERE TYPEOF(value) NOT IN ('OBJECT', 'ARRAY');
```

The column named **`this`** shows you what is being flattened for each row. This really only has value if you are doing a recursive flatten (otherwise, **`this`** will always show the input to the `FLATTEN` function).

If you look at the **`this`** column across from each phone number, you have a good idea of who the phone number is for based on their email address. But email addresses do not always include someone's name…so that is not the best indicator.

6.3.5 Include the `path` column in your query.

```
SELECT
   key, value, path
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true)
WHERE TYPEOF(value) NOT IN ('OBJECT', 'ARRAY');
```

The `path` column is essentially the dot notation syntax representation of how you got down to the value displayed. For a key of "phone," you can see that the path tells you whether it is a personal phone or a business phone. But that still doesn't solve the problem of who that phone number belongs to.

6.3.6 Include the `seq` column in your query.

© 2023 Snowflake Computing Inc. All Rights Reserved

```
SELECT
   seq, key, value
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true)
WHERE TYPEOF(value) NOT IN ('OBJECT', 'ARRAY');
```

Each row of JSON data is assigned a sequence number. So when you see multiple rows of output, you know that all rows with the same sequence value came out of the same row of data. Now it's easy to match up phone numbers with the owner - you just look for the first and last name with the same sequence number as the phone. And the path will tell you whether it is a personal phone, or a business phone.

In your output, it is very likely that the first six rows have a sequence value of 1, and the last six rows have a sequence value of 2. However, sequence numbers are not guaranteed to be displayed in order. So you should order your output by sequence number if you want to be sure that all the values from a single row are displayed together:

```
SELECT
   seq, key, value
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true)
WHERE IS_ARRAY(value)='false' AND IS_OBJECT(value)='false'
ORDER BY seq;
```

Now let's use what we've learned about FLATTEN to convert all of the data to a structured format.

6.3.7 Explode the data

Now we want to list ID, first name, last name, business phone number and email, and personal phone number and email. We need to explode the data to do that. But first we need to figure out the input into our FLATTEN function.

Start by running the query below:

```
SELECT
   info.*
FROM
   customers,
LATERAL FLATTEN(input=>info, RECURSIVE=>true) info
WHERE index IS NOT NULL;
```

Using what we learned earlier, we exploded all of the data but filtered to only find the rows where index is not null. This gives us all of the positions in the contact array.

If we look in the `path` column, we find `contact[0]` and `contact[1]`. Those are the values we want to input into our FLATTEN function.

© 2023 Snowflake Computing Inc. All Rights Reserved

6.3.8 Fetch ID, first name, last name and business details

Run the statement below:

```
SELECT
   ID,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name,
   business.value:phone::VARCHAR as business_phone,
   business.value:email::VARCHAR as business_email
FROM
   customers,
LATERAL FLATTEN(input=>info:contact[0]) business;
```

Fetching the ID, first and last names was simple. For the business details, we've inputted `contact[0]` into the FLATTEN function to access only the business details.

6.3.9 Fetch ID, first name, last name and personal details

Run the statement below:

```
SELECT
   ID,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name,
   personal.value:phone::VARCHAR as personal_phone,
   personal.value:email::VARCHAR as personal_email
FROM
   customers,
LATERAL FLATTEN(input=>info:contact[1]) personal;
```

Here we've fetched the personal details exactly as we fetched the business details. Now let's put it all together.

6.3.10 Fetch all contact details

```
SELECT
   ID,
   info:name.first::VARCHAR AS first_name,
   info:name.last::VARCHAR AS last_name,
   business.value:email::VARCHAR as business_email,
   business.value:phone::VARCHAR as business_phone,
   personal.value:email::VARCHAR as personal_email,
   personal.value:phone::VARCHAR as personal_phone
FROM
   customers,
LATERAL FLATTEN(input=>info:contact[0]) business,
LATERAL FLATTEN(input=>info:contact[1]) personal;
```

Here we simply used the FLATTEN function twice for each potential path. Our semi-structured data is now structured data.

© 2023 Snowflake Computing Inc. All Rights Reserved

6.4 Try it out!

In this section, you're going to query semi-structured data on your own. First you'll run a statement to create a short set of JSON data, then we'll walk you through some challenge exercises.

6.4.1 Setting the context

Set the context using the code shown below:

```
CREATE WAREHOUSE IF NOT EXISTS [login]_WH;
USE WAREHOUSE [login]_WH;

CREATE DATABASE IF NOT EXISTS [login]_DB;
USE DATABASE [login]_DB;

CREATE OR REPLACE SCHEMA WEATHER;
USE SCHEMA WEATHER;
```

6.4.2 Run the statement below to create the data you'll query

```
CREATE OR REPLACE TABLE weather_data
AS
SELECT
   parse_json($1) AS w
FROM VALUES
('{
  "data": {
    "observations": [
      {
        "air": {
          "dew-point": 8.2,
          "dew-point-quality-code": "1",
          "temp": 29.8,
          "temp-quality-code": "1"
        },
        "atmospheric": {
          "pressure": 10161,
          "pressure-quality-code": "1"
        },
        "dt": "2019-07-30T02:00:00",
        "sky": {
          "ceiling": 99999,
          "ceiling-quality-code": "9"
        },
        "visibility": {
          "distance": 999999,
          "distance-quality-code": "9"
        },
        "wind": {
          "direction-angle": 80,
          "direction-quality-code": "1",
          "speed-quality-code": "1",
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
        "speed-rate": 41
      }
    },
    {
      "air": {
        "dew-point": 8.2,
        "dew-point-quality-code": "1",
        "temp": 29.9,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10161,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T02:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 80,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 41
      }
    },
    {
      "air": {
        "dew-point": 6.4,
        "dew-point-quality-code": "1",
        "temp": 32.2,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10126,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T05:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 60,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 57
      }
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
    },
    {
      "air": {
        "dew-point": 6.4,
        "dew-point-quality-code": "1",
        "temp": 32.2,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10126,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T05:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 60,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 57
      }
    },
    {
      "air": {
        "dew-point": 5.1,
        "dew-point-quality-code": "1",
        "temp": 30.6,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10123,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T08:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 140,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 26
      }
    },
    {
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
      "air": {
        "dew-point": 5.1,
        "dew-point-quality-code": "1",
        "temp": 30.7,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10123,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T08:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 140,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 26
      }
    },
    {
      "air": {
        "dew-point": 3.5,
        "dew-point-quality-code": "1",
        "temp": 18.9,
        "temp-quality-code": "2"
      },
      "atmospheric": {
        "pressure": 10152,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T11:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 120,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 10
      }
    },
    {
      "air": {
        "dew-point": 3.5,
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
        "dew-point-quality-code": "1",
        "temp": 19,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10152,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T11:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 120,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 10
      }
    },
    {
      "air": {
        "dew-point": 5.5,
        "dew-point-quality-code": "1",
        "temp": 17.9,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10165,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T14:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 150,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 10
      }
    },
    {
      "air": {
        "dew-point": 5.5,
        "dew-point-quality-code": "1",
        "temp": 18,
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10165,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T14:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 150,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 10
      }
    },
    {
      "air": {
        "dew-point": 5,
        "dew-point-quality-code": "1",
        "temp": 13.3,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10163,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T17:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 140,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 5
      }
    },
    {
      "air": {
        "dew-point": 5,
        "dew-point-quality-code": "1",
        "temp": 13.4,
        "temp-quality-code": "1"
      },
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
      "atmospheric": {
        "pressure": 10163,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T17:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 140,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 5
      }
    },
    {
      "air": {
        "dew-point": 7.8,
        "dew-point-quality-code": "1",
        "temp": 13.8,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10177,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T20:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 160,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 15
      }
    },
    {
      "air": {
        "dew-point": 7.8,
        "dew-point-quality-code": "1",
        "temp": 13.9,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10177,
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T20:30:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 160,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 15
      }
    },
    {
      "air": {
        "dew-point": 9.4,
        "dew-point-quality-code": "1",
        "temp": 22.2,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10190,
        "pressure-quality-code": "1"
      },
      "dt": "2019-07-30T23:00:00",
      "sky": {
        "ceiling": 99999,
        "ceiling-quality-code": "9"
      },
      "visibility": {
        "distance": 999999,
        "distance-quality-code": "9"
      },
      "wind": {
        "direction-angle": 130,
        "direction-quality-code": "1",
        "speed-quality-code": "1",
        "speed-rate": 21
      }
    },
    {
      "air": {
        "dew-point": 9.4,
        "dew-point-quality-code": "1",
        "temp": 22.2,
        "temp-quality-code": "1"
      },
      "atmospheric": {
        "pressure": 10190,
        "pressure-quality-code": "1"
      },
```


© 2023 Snowflake Computing Inc. All Rights Reserved

```
        "dt": "2019-07-30T23:30:00",
        "sky": {
          "ceiling": 99999,
          "ceiling-quality-code": "9"
        },
        "visibility": {
          "distance": 999999,
          "distance-quality-code": "9"
        },
        "wind": {
          "direction-angle": 130,
          "direction-quality-code": "1",
          "speed-quality-code": "1",
          "speed-rate": 21
        }
      }
    ]
  },
  "station": {
    "USAF": "942340",
    "WBAN": 99999,
    "coord": {
      "lat": -16.25,
      "lon": 133.367
    },
    "country": "AS",
    "elev": 211,
    "id": "94234099999",
    "name": "DALY WATERS AWS"
  }
}');
```

6.4.3 Take a look at the following diagram to better understand the data you just created

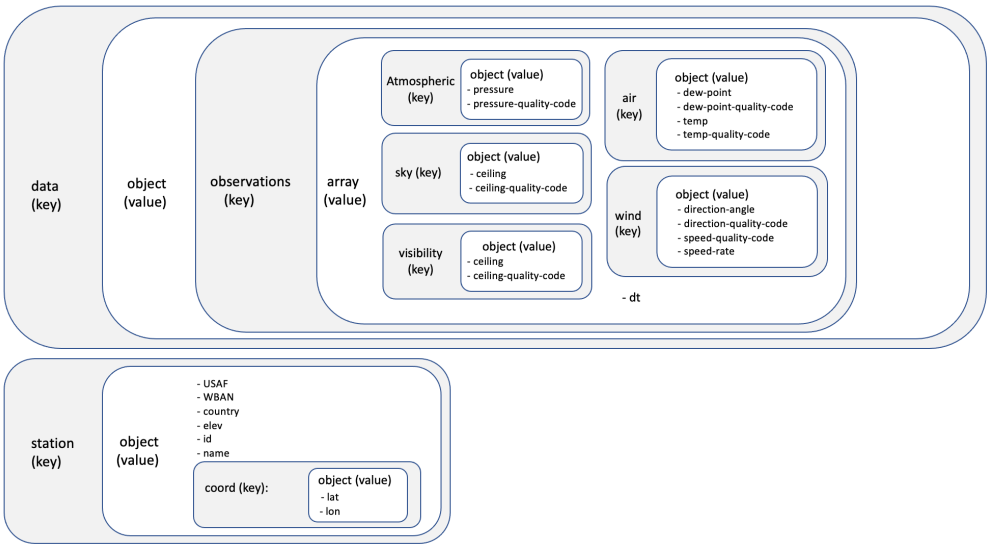


Figure 30: JSON diagram

© 2023 Snowflake Computing Inc. All Rights Reserved

In the previous step, you created a single document that is structured as shown in the diagram. As you can see, there are two top-level keys in the data: data and station.

Data (the first key) has a corresponding value that is an object. That object contains an array called observations. This array contains objects, each of which has six elements: a key-value pair called dt, and five key-value pairs named air, atmospheric, sky, visibility, and wind. With the exception of dt (which is a simple key-value pair), the value in each of the key-value pairs is an object that in turn contains a set of simple key-value pairs.

Station (the second key) has a corresponding value that in an object. That object consists of six key-value pairs and an object called coord. This object consists of two key-value pairs: lat and lon.

The general idea is that for a specific date at a specific station, a number of weather measurements (or observations) are taken. In this case, you have created a set of sixteen observations taken at different times throughout the day on Tuesday, July 30, 2019.

Now you'll be given a set of challenges based on what you've just learned. Working with semi-structured data can be tricky, and don't worry if you don't answer each challenge correctly. The point here is to try to apply the points in this lesson and to learn as much as you can. There is a solution at the end of this exercise if you need it.

6.4.4 Select USAF, WBAN, country, elev, id and name from station using dot notation.

6.4.5 Select average, max and min air temperature using LATERAL FLATTEN.

6.4.6 Select max and min atmospheric pressure using LATERAL FLATTEN.

6.4.7 Use either dot notation or FLATTEN LATERAL to fetch the dew point, atmospheric pressure, and wind speed rate for the 15th observation.

6.4.8 Check the solution just after the Key Takeaways if you need help. Otherwise congratulations! You've just completed this lab.

6.5 Key Takeaways

- JSON data, is stored in a column of data type VARIANT. This is true for all other forms of semi-structured data capable of being stored in Snowflake.

- While you can use VARIANT data types in comparisons or aggregations with other data types, your queries won't be as performant as they would be if you had cast the VARIANT columns to standard SQL data types.

- You can use either dot notation or LATERAL FLATTEN to query semi-structured data in an array.

- When working with arrays in nested JSON data, you will need to provide the index of the key-value pair in the array that you want so you can access its contents.

- You can use the FLATTEN function's output fields `seq`, `key`, `path`, `index`, `value` and **`this`** to figure out how your semi-structured data is structured.

- You can use FLATTEN multiple times in a FROM clause for each potential path.

© 2023 Snowflake Computing Inc. All Rights Reserved

6.6 Solution

```
--select USAF, WBAN, country, elev, id and name from station using dot notation
SELECT
        wd.w:station.USAF::VARCHAR AS USAF,
        wd.w:station.country::VARCHAR AS country,
        wd.w:station.elev::VARCHAR AS elev,
        wd.w:station.id::VARCHAR AS id,
        wd.w:station.name::VARCHAR AS name
 FROM
    weather_data wd;


-- select average, max and min air temperature using LATERAL FLATTEN
SELECT
          AVG(f.value:air.temp)::NUMBER(38,1) as avg_temp_c,
          MIN(f.value:air.temp) as min_temp_c,
          MAX(f.value:air.temp) as max_temp_c

FROM
    weather_data wd,
    LATERAL FLATTEN (input => w:data.observations) f  ;


-- select max and min atmospheric pressure using LATERAL FLATTEN
SELECT
          MAX(f.value:atmospheric.pressure)::NUMBER(38,1) as max_pressure,
          MIN(f.value:atmospheric.pressure)::NUMBER(38,1) as min_pressure
FROM
    weather_data wd,
    LATERAL FLATTEN (input => w:data.observations) f  ;


-- Use either dot notation or FLATTEN LATERAL to fetch the dew point, atmospheric
   pressure, and wind speed rate for the 15th observation
--dot notation
 SELECT
        wd.w:data.observations[15].air."dew-point"::NUMBER(38,1) AS dew_point,
        wd.w:data.observations[15].atmospheric.pressure::NUMBER(38,1) AS pressure,
        wd.w:data.observations[15].wind."speed-rate"::NUMBER(38,1) AS dew_point

 FROM
    weather_data wd;

-- flatten lateral
SELECT
        datapoint1.VALUE::NUMBER(38,1) AS dew_point,
        datapoint2.VALUE::NUMBER(38,1) AS pressure,
        datapoint3.VALUE::NUMBER(38,1) AS speed_rate
FROM
    weather_data wd,
    LATERAL FLATTEN (input => w:data.observations[15].air."dew-point", recursive
       => true) datapoint1,
    LATERAL FLATTEN (input => w:data.observations[15].atmospheric.pressure,
       recursive => true) datapoint2,
```

© 2023 Snowflake Computing Inc. All Rights Reserved

```
    LATERAL FLATTEN (input => w:data.observations[15].wind."speed-rate", recursive
       => true) datapoint3;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

7 Introduction to Monitoring Usage and Billing

The purpose of this lab is to familiarize you with the Snowflake database, the schemas in the database, and how you can use that data to monitor how users are using the objects in your system and what the associated costs are with that usage.

Learning Objectives:

- Monitor usage and billing with the ACCOUNT_USAGE schema
- Determine which warehouses do not have resource monitors activated for them
- Determine the most expensive queries from the last 30 days
- Determine the top 10 queries with the most spillage to remote storage

HOW TO COMPLETE THIS LAB

As the workbook PDF may have useful diagrams, we recommend that you read the instructions from the workbook PDF. In order to execute the code presented in each step, use the SQL code file that was provided for this lab.

OPENING THE SQL FILE

To load the SQL file, select **Worksheets** in the left navigation bar and click the ellipses in the upper-right corner of your Snowsight window. Select **Create Worksheet from SQL File** from the drop-down menu. Navigate to the lab file for this lab and load it.

Snowflake Database

Snowflake provides a system-defined, read-only shared database named `SNOWFLAKE` that contains metatdata and historical usage data about the objects in your organization and account.

The purpose of the database is to allow you to monitor object usage metrics as well as the costs associated with that usage so you can make any adjustments needed to get the most for the credits being spent.

There are many schemas in the `SNOWFLAKE` database, but the one we are going to concentrate on is `ACCOUNT_USAGE`.



We have given you access to `ACCOUNT_USAGE` for learning purposes. However, by default only users with the `ACCOUNTADMIN` can access the `SNOWFLAKE` database and schemas, or perform queries on the views. Privileges on the database can be granted to other roles in your account to allow other users to access the objects.



Links to more information about the `SNOWFLAKE` database, and its other schemas, can be found here:
Click here for Snowflake Database (https://docs.snowflake.com/en/sql-reference/snowflake-db.html)

© 2023 Snowflake Computing Inc. All Rights Reserved

Let's get started!

7.1 Monitor Usage and Billing with the `ACCOUNT_USAGE` schema

The `ACCOUNT_USAGE` schema supports usage and billing monitoring because it exposes a number of secure views that display data related to object usage history, grants to roles and users, data loading history, metering history, storage history, task history, users, roles and more.

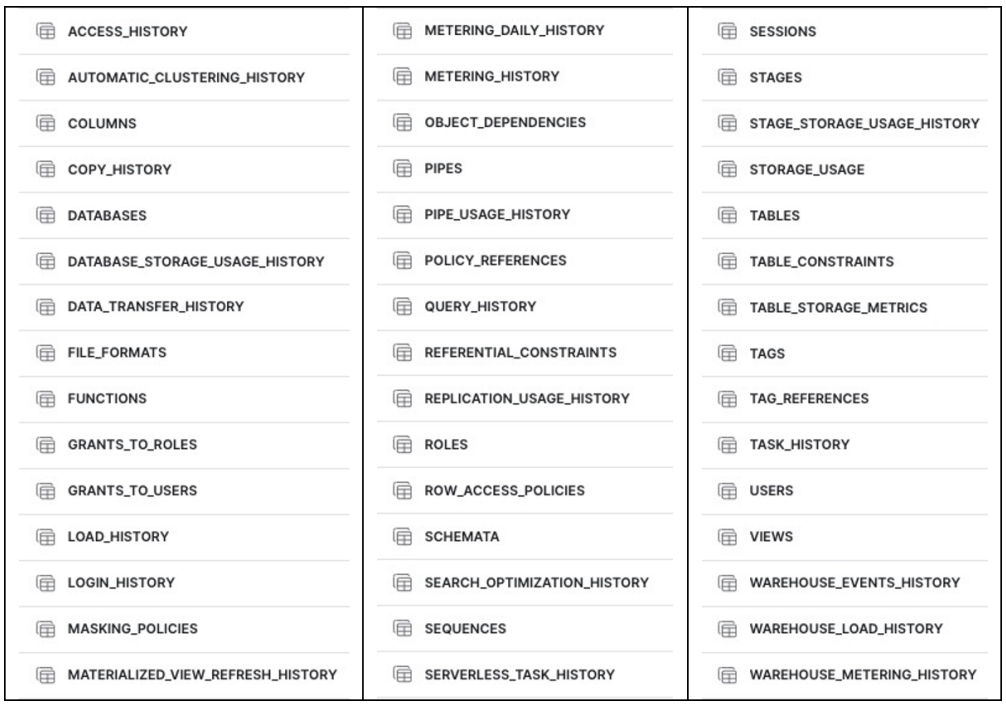


Figure 31: Secure views present in the SNOWFLAKE.ACCOUNT_USAGE schema

The main approach to writing queries that will help you monitor usage and billing is to look at the views that appear relevant and then run through the column list to see if they appear relevant to your goal. As there are many views in each schema, some with a lot of columns, it will take time and experience working with the schemas to become sufficiently familiar with them to get the answers you want.

7.1.1 Set your context:

For this lab, you will be using `SNOWFLAKE.ACCOUNT_USAGE` as your database and schema.

```
USE ROLE TRAINING_ROLE;

CREATE WAREHOUSE IF NOT EXISTS [login]_WH INITIALLY_SUSPENDED=TRUE;
USE WAREHOUSE [login]_WH;

USE SCHEMA SNOWFLAKE.ACCOUNT_USAGE;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

7.1.2 Examine credit usage by warehouse.

Monitoring credit consumption for specific objects is a classic use of data in the `ACCOUNT_USAGE` schema. Here is an example of two queries that use the `WAREHOUSE_METERING_HISTORY` view:

```
-- Credits used (all time = past year)
SELECT WAREHOUSE_NAME
      ,SUM(CREDITS_USED_COMPUTE) AS CREDITS_USED_COMPUTE_SUM
  FROM ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
 GROUP BY 1
 ORDER BY 2 DESC;

-- Credits used (past N days/weeks/months)
SELECT WAREHOUSE_NAME,
       SUM(CREDITS_USED_COMPUTE) AS CREDITS_USED_COMPUTE_SUM
  FROM ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
 WHERE START_TIME >= DATEADD(DAY, -7, CURRENT_TIMESTAMP())  // Past 7 days
 GROUP BY 1
 ORDER BY 2 DESC;
```



Notices the `GROUP BY 1` and `ORDER BY 2` in the query above. If your column names are long, you can group or order by the column position, rather than the column name.

These queries enable you to determine if there are specific warehouses that are consuming more credits than the others. You can then drill into questions such as: Should they be consuming that quantity of credits? Are there specific warehouses that are consuming more credits than anticipated?

In the event a warehouse is consuming too many credits you could take action to rectify the sitution. Depending on what the warehouse is being used for you could consider modifying the auto-suspend policy or the scaling policy, checking the data loading history to see if efficient practices are being used, or analyzing the size and efficiency of queries being run on the warehouse, or adding a resource monitor to the warehouse.

Below is a list of the columns in this view.



Since Snowflake is continually adding functionality, this could be somewhat different from what you'll see in the Snowsight User Interface.

© 2023 Snowflake Computing Inc. All Rights Reserved

NAME ↑	TYPE	NULLABLE	DEFAULT
CREDITS_USED	NUMBER(38,9)	Yes	NULL
CREDITS_USED_CLOUD_SERVICES	NUMBER(38,9)	Yes	NULL
CREDITS_USED_COMPUTE	NUMBER(38,9)	Yes	NULL
END_TIME	TIMESTAMP_LTZ(0)	Yes	NULL
START_TIME	TIMESTAMP_LTZ(0)	Yes	NULL
WAREHOUSE_ID	NUMBER(38,0)	Yes	NULL
WAREHOUSE_NAME	VARCHAR(16777216)	Yes	NULL

Figure 32: Warehouse Metering History columns

7.1.3 Determining warehouses without resource monitors

If you have warehouses that are using too many credits, you can put resource monitors on them.

The query below identifies all warehouses without resource monitors in place. Resource monitors provide the ability to set limits on credits consumed against a warehouse during a specific time interval or date range. This can help prevent certain warehouses from unintentionally consuming more credits than typically expected.

```
SHOW WAREHOUSES;

SELECT "name" AS WAREHOUSE_NAME
      ,"size" AS WAREHOUSE_SIZE
  FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
 WHERE "resource_monitor" = 'null';
```

You shouldn't find any warehouses without resource monitors because all of them have at least one. So, your result should be 0. However, if you have not put a resource monitor on your own warehouse ([login]_WH), then that warehouse may appear in the results.

7.2 Billing Metrics

Billing metrics is all about analyzing what you've been billed in the past so you can determine if there is a way to lower costs in the future.

7.2.1 Most expensive queries from the last 30 days

The query below analyzes queries that are potentially too expensive by ordering the most expensive queries from the last 60 days. It takes into account the warehouse size, assuming that a 1 minute query on a larger warehouse is more expensive than a 1 minute query on a smaller warehouse.

© 2023 Snowflake Computing Inc. All Rights Reserved

```
WITH WAREHOUSE_SIZE AS
(
     SELECT WAREHOUSE_SIZE, "CREDITS/HOUR"
       FROM (
              SELECT 'XSMALL' AS WAREHOUSE_SIZE, 1 AS "CREDITS/HOUR"
              UNION ALL
              SELECT 'SMALL' AS WAREHOUSE_SIZE, 2 AS "CREDITS/HOUR"
              UNION ALL
              SELECT 'MEDIUM' AS WAREHOUSE_SIZE, 4 AS "CREDITS/HOUR"
              UNION ALL
              SELECT 'LARGE' AS WAREHOUSE_SIZE, 8 AS "CREDITS/HOUR"
              UNION ALL
              SELECT 'XLARGE' AS WAREHOUSE_SIZE, 16 AS "CREDITS/HOUR"
              UNION ALL
              SELECT '2XLARGE' AS WAREHOUSE_SIZE, 32 AS "CREDITS/HOUR"
              UNION ALL
              SELECT '3XLARGE' AS WAREHOUSE_SIZE, 64 AS "CREDITS/HOUR"
              UNION ALL
              SELECT '4XLARGE' AS WAREHOUSE_SIZE, 128 AS "CREDITS/HOUR"
            )
),
QUERY_HISTORY AS
(
     SELECT QH.QUERY_ID
           ,QH.QUERY_TEXT
           ,QH.USER_NAME
           ,QH.ROLE_NAME
           ,QH.EXECUTION_TIME
           ,QH.WAREHOUSE_SIZE
      FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY QH
     WHERE START_TIME > DATEADD(month,-2,CURRENT_TIMESTAMP())
)

SELECT QH.QUERY_ID
      ,'https://' || current_account() ||
         '.snowflakecomputing.com/console#/monitoring/queries/detail?queryId='
           ||QH.QUERY_ID AS QU
      ,QH.QUERY_TEXT
      ,QH.USER_NAME
      ,QH.ROLE_NAME
      ,QH.EXECUTION_TIME as EXECUTION_TIME_MILLISECONDS
      ,(QH.EXECUTION_TIME/(1000)) as EXECUTION_TIME_SECONDS
      ,(QH.EXECUTION_TIME/(1000*60)) AS EXECUTION_TIME_MINUTES
      ,(QH.EXECUTION_TIME/(1000*60*60)) AS EXECUTION_TIME_HOURS
      ,WS.WAREHOUSE_SIZE
      ,WS."CREDITS/HOUR"
      ,(QH.EXECUTION_TIME/(1000*60*60))*WS."CREDITS/HOUR" as
         RELATIVE_PERFORMANCE_COST

FROM QUERY_HISTORY QH
JOIN WAREHOUSE_SIZE WS ON WS.WAREHOUSE_SIZE = upper(QH.WAREHOUSE_SIZE)
ORDER BY RELATIVE_PERFORMANCE_COST DESC
LIMIT 200;
```

© 2023 Snowflake Computing Inc. All Rights Reserved

This query gives you the chance to evaluate expensive queries and take some action. For example, you could look at the query profile, contact the user who executed the query, or take action to optimize these queries.

Below is a list of the columns in this secure view, SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY.

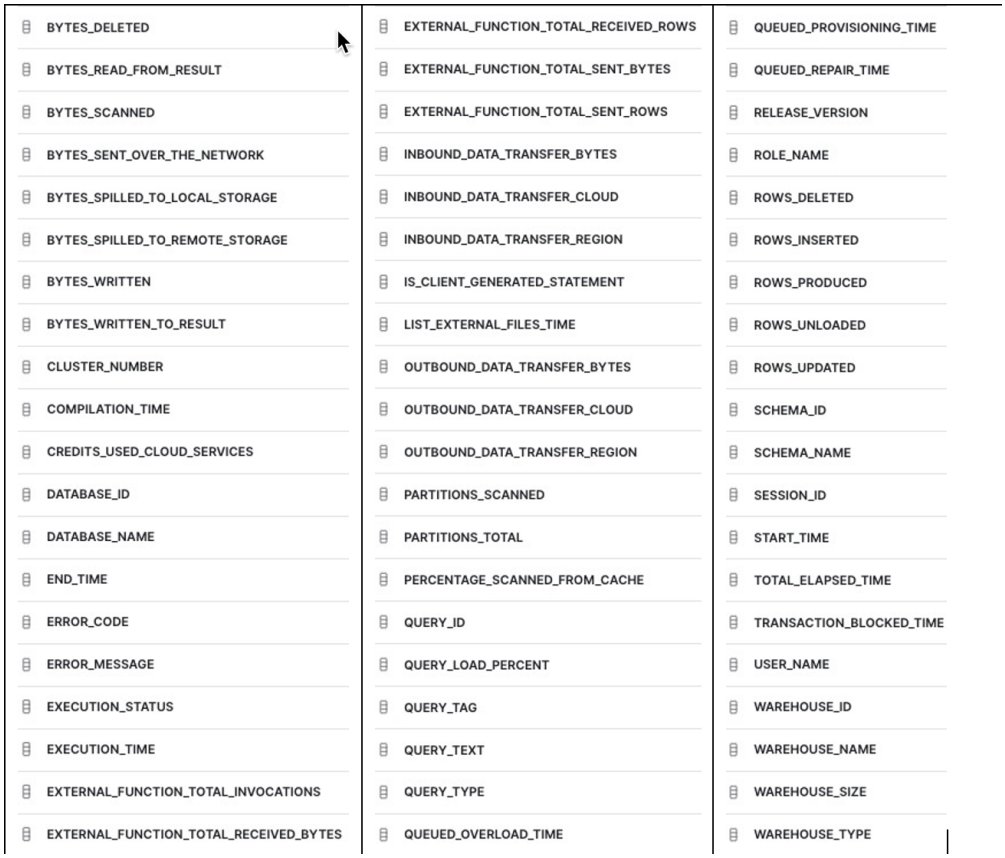


Figure 33: QUERY_HISTORY view columns

7.2.2 Top 10 Queries With The Most Spillage to Remote Storage

Another way to evaluate the cost of queries is to see if they are spilling to remote storage. The query below allows you to do that.

```
SELECT query_id, substr(query_text, 1, 50) partial_query_text, user_name,
   warehouse_name, warehouse_size,
       BYTES_SPILLED_TO_REMOTE_STORAGE, start_time, end_time,
          total_elapsed_time/1000 total_elapsed_time
FROM   snowflake.account_usage.query_history
WHERE  BYTES_SPILLED_TO_REMOTE_STORAGE > 0
AND start_time::date > dateadd('days', -45, current_date)
ORDER BY BYTES_SPILLED_TO_REMOTE_STORAGE DESC
LIMIT 10;
```

Because you are in a training account, it is likely that you won't find any queries that have remote spillage. However, in a production environment you will most likely encounter queries that have some remote spillage.

© 2023 Snowflake Computing Inc. All Rights Reserved

This query also provides the warehouse name and size. Once you identify the queries that are spilling to remote storage, you can take action to ensure they are run on larger warehouses with more local storage and memory.

7.3 Key Takeaways

- There are many views in each schema of the Snowflake database, some with a lot of columns. It will take time and experience working with the schemas to become sufficiently familiar with them to get the answers you want.

- Resource monitors provide the ability to set limits on credits consumed against a warehouse during a specific time interval or date range.

- The Snowflake database enables you to determine where you have high credit consumption so you can ask pertinent questions as to why that is occurring. You can use this information to help you consider steps to bring down costs.

© 2023 Snowflake Computing Inc. All Rights Reserved